Universidad de las Ciencias Informáticas
Facultad de Tecnologías Interactivas

Videojuego educativo para la enseñanza de los Objetivos de Desarrollo Sostenible

Trabajo de diploma para optar por el título de Ingeniero en Ciencias Informáticas

Autor: Ruslan Alexei Martinez Campos

Tutores: P.T., Ing. Juan Antonio Plasencia Soler, Dr. C.

Ing. Oscar de Jesús Pacheco del Pino

La Habana, Marzo de 2026

Año del Centenario del Comandante en Jefe Fidel Castro Ruz

Escribe aquí tu pensamiento o frase inspiradora
Autor del pensamiento

Dedicatoria

Escribe aquí tu dedicatoria

Agradecimientos

Escribe aquí tus agradecimientos

Declaración de autoría

Declaramos ser autores de la presente tesis y reconocemos a la Universidad de las Ciencias Informáticas los derechos patrimoniales sobre esta, con carácter exclusivo.

Para que así conste firmamos la presente a los ____ días del mes de _____ del año _____.

Ruslan Alexei Martinez Campos
Autor

P.T., Ing. Juan Antonio Plasencia Soler, Dr. C.
Tutor

Ing. Oscar de Jesús Pacheco del Pino
Tutor

Resumen

El desconocimiento generalizado sobre los Objetivos de Desarrollo Sostenible (ODS) de las Naciones Unidas dificulta la participación activa en la Agenda 2030 y limita la toma de decisiones informadas. Promover un conocimiento profundo de estos 17 objetivos es crucial para mejorar la calidad de vida y fomentar el desarrollo equitativo. En este contexto, los videojuegos educativos —especialmente aquellos basados en simulación y aprendizaje interactivo— emergen como una herramienta eficaz, generando experiencias inmersivas que facilitan el aprendizaje significativo gracias a su capacidad para integrar narrativas, retroalimentación inmediata y gamificación. El presente documento describe el proceso completo de desarrollo de una solución basada en videojuegos, abarcando desde la selección del motor (Godot) y la obtención rigurosa de requisitos funcionales y no funcionales mediante historias de usuario, hasta el diseño de interfaces, la construcción de una arquitectura escalable y la explicación de componentes esenciales (tablero, preguntas, mecanismos de avance). Finalmente, se detalla la aplicación de pruebas de software, en particular las pruebas de aceptación, para validar la calidad y eficacia de la solución propuesta.

ABSTRACT

The widespread lack of knowledge about the United Nations' Sustainable Development Goals (SDGs) limits public engagement with the 2030 Agenda and hinders informed decision-making. Promoting a deeper understanding of the 17 goals is crucial for improving quality of life and fostering equitable development. In this context, educational video games—especially those based on simulation and interactive learning—emerge as an effective tool, offering immersive experiences that support meaningful learning by integrating strong narratives, immediate feedback, and gamified challenges. This document outlines the complete development process of a video-game-based solution, from selecting the development engine (Godot) and rigorously gathering functional and non-functional requirements, to designing graphical interfaces and establishing a scalable software architecture. Key game components are described, and finally, the execution of software testing, particularly acceptance tests, is presented to validate the proposed solution's quality and effectiveness.

Palabras clave: ODS, Agenda 2030, videojuegos educativos, Godot Engine, exportación multiplataforma..

Índice general

Lista de tareas pendientes XIV

Introducción 1

1 Fundamentos y referentes teórico-metodológicos sobre el objeto de estudio 4

1.1 Fundamentos teóricos de los videojuegos educativos . . . 4

1.1.1 Antecedentes del aprendizaje mediante el juego . . . 4

1.1.2 Fundamentos del Aprendizaje Basado en Juegos (Game-Based Learning) . . . 5

1.1.3 Gamificación como estrategia educativa . . . 6

1.2 Gamificación en la educación para el desarrollo sostenible . . . 7

1.2.1 Antecedentes internacionales . . . 7

1.2.2 Antecedentes nacionales . . . 9

1.2.3 Estudio de soluciones homólogas . . . 9

1.3 Análisis del estado actual del juego de mesa Go Goals! . . . 10

1.3.1 Descripción del juego Go Goals! en su formato original . . . 11

1.3.2 Diagnóstico: fortalezas y limitaciones del formato físico . . . 12

1.4 Metodologías y herramientas para el desarrollo de Go Goals! Digital . . . 13

1.4.1 Metodología de desarrollo de software . . . 13

1.4.2 Herramienta CASE . . . 13

1.4.3 Lenguaje de modelado . . . 14

1.4.4 Motor de videojuegos . . . 15

1.4.5 Lenguaje de programación . . . 16

1.4.6 Herramienta de diseño de activos gráficos . . . 16

1.4.7 Sistema de control de versiones . . . 17

1.4.8 Software de gestión de proyectos . . . 17

Conclusiones del capítulo . . . 18

2 Planificación, diseño e implementación de la solución propuesta 20

2.1 Descripción de la propuesta de solución . . . 20

2.2 Planificación . . . 21

2.2.1 Requisitos funcionales 22
2.2.2 Requisitos no funcionales 23
2.2.3 Historias de usuario 24
2.2.4 Estimación de tiempo de desarrollo 35
2.2.5 Plan de iteraciones 36
2.3 Diseño 37
2.3.1 Arquitectura utilizada 38
2.3.2 Prototipos de interfaces gráficas 40
2.3.3 Tarjetas CRC 41
2.4 Implementación 44
2.4.1 Estándares de codificación 44
2.4.2 Patrones de diseño utilizados 45
2.4.3 Tareas de ingeniería 47
Conclusiones del capítulo 53

3 Resultados **54**

Conclusiones **55**

Recomendaciones **56**

Referencias bibliográficas **57**

Apéndices **61**

A Proyectos y Avales **62**

B Otro reporte **63**
B.1 Otra sección de muestra 63
B.2 Otro ensamble Industrial 64

Índice de figuras

1.1 Relación entre los marcos teóricos empleados en el diseño del videojuego educativo (elaboración propia) . . . 7
1.2 Tablero oficial del juego de mesa Go Goals! de las Naciones Unidas (Naciones Unidas, 2020) 8
1.3 Estado actual de los recursos educativos digitales sobre ODS en Cuba (elaboración propia) . 9
1.4 Mecánica de tablero digital interactivo en la serie comercial *Mario Party (elaboración propia)* . . . 11
1.5 Fases de Extreme Programming aplicadas al desarrollo de *Go Goals! Digital (elaboración propia)* (elaboración propia) . . . 14
1.6 Comparativa de motores de videojuegos por criterios del proyecto (elaboración propia) . . . 16
1.7 Diseño vectorial del tablero hexagonal de *Go Goals! Digital (elaboración propia)* en Inkscape (elaboración propia) . . . 17
1.8 Tablero de gestión de tareas en GitHub Projects para *Go Goals! Digital (elaboración propia)* (elaboración propia) . . . 18

2.1 Vista general del videojuego educativo *Go Goals! Digital (elaboración propia)* (elaboración propia) . . . 21
2.2 SceneTree y Autoloads del videojuego *Go Goals! Digital (elaboración propia)* en Godot Engine (elaboración propia) . . . 39
2.3 Arquitectura multi-patrón del videojuego educativo *Go Goals! Digital (elaboración propia)* (elaboración propia) . . . 39
2.4 Prototipo de la pantalla principal del tablero de juego (elaboración propia) . . . 40
2.5 Prototipo del panel de preguntas educativas por ODS (elaboración propia) . . . 41
2.6 Prototipo de la pantalla de estadísticas de aprendizaje (elaboración propia) . . . 41
2.7 Estructura de archivos del proyecto *Go Goals! Digital (elaboración propia)* (elaboración propia) . . . 45
2.8 Patrón Bus de Eventos implementado mediante señales globales en Godot (elaboración propia) 46

Índice de tablas

1.1 Ejemplos de preguntas por nivel cognitivo según ODS (elaboración propia) 8
1.2 Videojuegos educativos y de tablero relacionados a la sostenibilidad (elaboración propia) . . 10
1.3 Componentes del juego de mesa Go Goals! (elaboración propia) 11
1.4 Diagnóstico del estado actual de Go Goals! (formato físico) (elaboración propia) 12
1.5 Comparativa ampliada de motores de videojuegos según criterios del proyecto (elaboración propia) 15

2.1 Requisitos funcionales del videojuego Go Goals! Digital (elaboración propia) 22
2.2 Requisitos no funcionales del videojuego Go Goals! Digital (elaboración propia) 23
2.3 Historia de usuario 1: Inicio de nueva partida educativa (elaboración propia) 25
2.4 Historia de usuario 2: Visualización del tablero hexagonal (elaboración propia) 25
2.5 Historia de usuario 3: Lanzamiento de dado virtual (elaboración propia) 26
2.6 Historia de usuario 4: Movimiento animado de ficha (elaboración propia) 26
2.7 Historia de usuario 5: Efecto de escalera y serpiente (elaboración propia) 27
2.8 Historia de usuario 6: Preguntas educativas sobre ODS (elaboración propia) 27
2.9 Historia de usuario 7: Evaluación y retroalimentación de respuestas (elaboración propia) . . 28
2.10 Historia de usuario 8: Gestión de turnos automática (elaboración propia) 28
2.11 Historia de usuario 9: Condición de victoria (elaboración propia) 29
2.12 Historia de usuario 10: Estadísticas de aprendizaje (elaboración propia) 29
2.13 Historia de usuario 11: Menú principal y pantalla de instrucciones (elaboración propia) . . . 30
2.14 Historia de usuario 12: Configuración de audio (elaboración propia) 30
2.15 Historia de usuario 13: Personalización de jugadores (elaboración propia) 31
2.16 Historia de usuario 14: Visualización de información del ODS (elaboración propia) 31
2.17 Historia de usuario 15: Temporizador de respuesta (elaboración propia) 32
2.18 Historia de usuario 16: Modo de un jugador (elaboración propia) 32
2.19 Historia de usuario 17: Pantalla de clasificación al finalizar (elaboración propia) 33
2.20 Historia de usuario 18: Notificación visual del turno activo (elaboración propia) 33
2.21 Historia de usuario 19: Accesibilidad de la interfaz (elaboración propia) 34
2.22 Historia de usuario 20: Pantalla de créditos (elaboración propia) 34
2.23 Historia de usuario 21: Exportación multiplataforma de escritorio (elaboración propia) . . . 35

2.24 Puntos de estimación por historia de usuario (elaboración propia) 36
2.25 Plan de iteraciones de desarrollo (elaboración propia) 37
2.26 Tarjeta CRC GameData (Autoload) (elaboración propia) 42
2.27 Tarjeta CRC GameEvents (Bus de Eventos) (elaboración propia) 42
2.28 Tarjeta CRC GameState (elaboración propia) 42
2.29 Tarjeta CRC GameManager (Controlador) (elaboración propia) 43
2.30 Tarjeta CRC BoardConfig (elaboración propia) 43
2.31 Tarjeta CRC Board (Entidad) (elaboración propia) 43
2.32 Tarjeta CRC Tile (Entidad) (elaboración propia) 43
2.33 Tarjeta CRC Player (Entidad) (elaboración propia) 44
2.34 Tarjeta CRC AudioManager (Autoload) (elaboración propia) 44
2.35 Tarjeta CRC RecordsManager (Autoload) (elaboración propia) 44
2.36 Tarea de ingeniería 1: Diseño e implementación del tablero hexagonal (elaboración propia) . 47
2.37 Tarea de ingeniería 2: Implementación del menú principal (elaboración propia) 48
2.38 Tarea de ingeniería 3: Lógica del dado y movimiento de fichas (elaboración propia) 48
2.39 Tarea de ingeniería 4: Escaleras, serpientes y gestión de turnos (elaboración propia) 49
2.40 Tarea de ingeniería 5: Sistema de preguntas y evaluación de respuestas (elaboración propia) . 49
2.41 Tarea de ingeniería 6: Estadísticas de aprendizaje (elaboración propia) 50
2.42 Tarea de ingeniería 7: Personalización de jugadores (elaboración propia) 50
2.43 Tarea de ingeniería 8: Panel informativo de ODS (elaboración propia) 51
2.44 Tarea de ingeniería 9: Notificaciones de turno activo (elaboración propia) 51
2.45 Tarea de ingeniería 10: Inteligencia Artificial oponente (elaboración propia) 52
2.46 Tarea de ingeniería 11: Exportación y optimización multiplataforma (elaboración propia) . . 52
2.47 Tarea de ingeniería 12: Configuración de sistema y accesibilidad (elaboración propia) 53

Lista de algoritmos

Lista de códigos fuentes

Lista de tareas pendientes

Introducción

La sostenibilidad se define como la capacidad de satisfacer las necesidades del presente sin comprometer la capacidad de las futuras generaciones para satisfacer sus propias necesidades, abarcando la interrelación entre el desarrollo económico, la inclusión social y la protección del medio ambiente (Comisión Mundial sobre el Medio Ambiente y el Desarrollo, 1987). Este concepto implica un enfoque holístico que busca un equilibrio entre estos tres pilares para garantizar un futuro viable para todos. Las dimensiones de la sostenibilidad se dividen comúnmente en tres categorías principales: la dimensión económica, centrada en el crecimiento y la eficiencia de recursos a largo plazo (Stiglitz; Sen y Fitoussi, 2018); la dimensión social, que promueve la equidad, la inclusión y el bienestar humano (Naciones Unidas, 2015); y la dimensión ambiental, enfocada en la conservación de la biodiversidad y la gestión responsable de los recursos naturales (European Union, 2024).

La educación para el desarrollo sostenible es fundamental en la promoción de un futuro equitativo. Este tipo de educación no solo busca transmitir conocimientos, sino que también fomenta habilidades y actitudes que permiten a las personas tomar decisiones informadas y responsables. La Educación para el Desarrollo Sostenible (EDS) es un enfoque integral que empodera a los ciudadanos para participar activamente en la construcción de sociedades justas y sostenibles (UNESCO, 2017; UNESCO, 2020). Además, la EDS contribuye directamente a la consecución de los Objetivos de Desarrollo Sostenible (ODS), especialmente a la meta 4.7 del ODS 4, garantizando que las personas adquieran los conocimientos teóricos y prácticos necesarios para promover estilos de vida sostenibles (Naciones Unidas, 2015; UNESCO, 2020).

La educación es un derecho humano fundamental y un motor clave para el desarrollo sostenible. Sin embargo, persisten brechas críticas: en 2021, 244 millones de niños y jóvenes estaban sin escolarizar, cifra que refleja desigualdades arraigadas en geografía y nivel socioeconómico (Global Education Monitoring Report Team, 2022). Estas brechas se intensifican en crisis humanitarias y se vieron exacerbadas por la pandemia de COVID-19, donde 463 millones de niños no tuvieron acceso a educación a distancia (UNICEF, 2020). En América Latina, la falta de dispositivos y conectividad afectó significativamente la continuidad educativa (Banco Mundial, 2025). Por ello, abordar estas brechas resulta esencial para lograr el ODS 4: una educación inclusiva, equitativa y de calidad para todos.

En Cuba, el sistema educativo ha sido históricamente un pilar del desarrollo social. El 99,7 % de su población adulta está alfabetizada (UNESCO, 2023), y la educación es gratuita en todos los niveles tal como lo establece la Constitución de la República (República de Cuba, 2019). No obstante, persisten desafíos como la escasez de materiales didácticos digitales, la obsolescencia tecnológica y las limitaciones

de conectividad que afectan la calidad educativa en contextos de crisis económica (Banco Mundial, 2023; Mesa-Lago, 2022). El Primer Informe Nacional Voluntario de Cuba (Grupo Nacional para la Implementación de la Agenda 2030, 2021) destaca la necesidad de continuar mejorando la infraestructura y el acceso a tecnologías actualizadas para asegurar una educación de calidad alineada con los estándares internacionales.

Los videojuegos se han posicionado como herramientas pedagógicas innovadoras para abordar desafíos de sostenibilidad. Ejemplos como *Eco*, que simula ecosistemas virtuales (Wu y J. J. Lee, 2020), o *Fate of the World*, enfocado en políticas climáticas globales (Flood; Cradock-Henry y Blackett, 2024), demuestran cómo estas herramientas pueden fomentar el pensamiento crítico y la comprensión de sistemas complejos. Los juegos de mesa, por su parte, constituyen una categoría pedagógica probada durante décadas: ofrecen entornos estructurados que desarrollan habilidades cognitivas y sociales de manera accesible (Martínez; Gimenes y Lambert, 2023). En ese contexto, las Naciones Unidas desarrollaron el juego de mesa *Go Goals!* como herramienta pedagógica oficial para introducir los diecisiete ODS a niños de entre ocho y diez años (Naciones Unidas, 2020), consolidándolo como un referente internacional del aprendizaje basado en juegos aplicado a la sostenibilidad.

El conocimiento de los ODS conlleva múltiples beneficios: alineación con la Agenda Global, mejora en la toma de decisiones y fomento de la responsabilidad social y ambiental (R3crea, 2022). Por el contrario, su desconocimiento genera desalineación con tendencias internacionales, uso inadecuado de recursos y menor preparación para enfrentar desafíos globales como el cambio climático (EcologíaVerde, 2024). En el contexto nacional, y específicamente en la Universidad de las Ciencias Informáticas, se identifica la necesidad de herramientas que contribuyan eficazmente a la promoción y enseñanza de estos objetivos.

No se ha encontrado ningún videojuego educativo desarrollado en Cuba que adapte las mecánicas del juego de mesa *Go Goals!* de las Naciones Unidas para la enseñanza explícita de los ODS. Aprovechando el potencial pedagógico de los videojuegos educativos y la estructura probada del juego de mesa original (Naciones Unidas, 2020), la presente investigación se propone desarrollar una versión digital de dicha experiencia: una herramienta interactiva, accesible y atractiva que contribuya a la promoción de los Objetivos de Desarrollo Sostenible, superando las barreras del formato físico.

Marco Teórico

El presente marco teórico se orienta a fundamentar y contextualizar la investigación en torno al problema científico planteado. En este sentido, se establece como objeto de estudio los videojuegos educativos como herramientas para la promoción de los ODS, focalizando el campo de acción en el proceso de desarrollo de un videojuego educativo basado en la adaptación digital del juego de mesa *Go Goals!*, por considerarlo un instrumento idóneo para fomentar el conocimiento y la sensibilización sobre los objetivos globales de la Agenda 2030. El objetivo general del estudio es desarrollar un videojuego educativo basado en la adaptación digital del juego de mesa *Go Goals!* que contribuya a la promoción de los Objetivos de Desarrollo Sostenible. Para alcanzar este propósito, se han definido tres objetivos específicos:

1. Establecer los referentes teórico-metodológicos relacionados con la promoción de los Objetivos de

Desarrollo Sostenible mediante videojuegos educativos.

2. Realizar el análisis, diseño e implementación del videojuego educativo basado en *Go Goals!*.
3. Validar el funcionamiento del videojuego a través de la aplicación de métricas de calidad y pruebas de software.

Situación problemática: El juego de mesa *Go Goals!* de las Naciones Unidas, reconocido internacionalmente como recurso pedagógico para la enseñanza de los ODS a niños de 8 a 10 años, presenta limitaciones estructurales inherentes a su formato físico que restringen su acceso, escalabilidad, rejugabilidad y capacidad de evaluación del aprendizaje.

Problema científico: ¿Cómo desarrollar un videojuego educativo basado en el juego de mesa *Go Goals!* que contribuya a la promoción de los Objetivos de Desarrollo Sostenible?

Métodos de investigación: La metodología de investigación se fundamenta en un enfoque integral que combina métodos empíricos, como la observación apoyada en el análisis comparativo de otros videojuegos educativos, con métodos teóricos, como la modelación y el análisis-síntesis, garantizando así una comprensión del fenómeno desde múltiples perspectivas.

Estructura capitular: El presente trabajo se estructura en tres capítulos. El **Capítulo I** establece los referentes teórico-metodológicos sobre el objeto de estudio, sistematizando los fundamentos del aprendizaje basado en juegos, la gamificación educativa y las tecnologías seleccionadas. El **Capítulo II** aborda el análisis, diseño e implementación del videojuego, presentando los artefactos de ingeniería de requisitos, la arquitectura de la solución y los prototipos de interfaces gráficas. El **Capítulo III** valida la solución propuesta mediante la aplicación de pruebas de software y métricas de calidad.

CAPÍTULO 1

Fundamentos y referentes teórico-metodológicos sobre el objeto de estudio

El presente capítulo tiene como objetivo sistematizar los fundamentos teórico-metodológicos que sustentan el desarrollo de un videojuego educativo para la promoción de los Objetivos de Desarrollo Sostenible. Se aborda, en primer lugar, el marco conceptual de los videojuegos educativos y el aprendizaje basado en juegos; en segundo lugar, se analizan los antecedentes de la gamificación aplicada a la educación para el desarrollo sostenible; en tercer lugar, se describe y diagnostica el estado actual del juego de mesa "Go Goals!" como objeto de estudio; y finalmente, se justifican las tecnologías y herramientas seleccionadas para la implementación de la solución propuesta. El capítulo cierra con las conclusiones que fundamentan las decisiones adoptadas en la investigación.

1.1. Fundamentos teóricos de los videojuegos educativos

El presente epígrafe sistematiza los principios teóricos y metodológicos que sustentan la incorporación de videojuegos educativos como recursos de aprendizaje significativo, especialmente cuando se diseñan a partir de dinámicas lúdicas tradicionales como los juegos de mesa y se orientan a la enseñanza de conceptos vinculados con la sostenibilidad. Su objetivo es establecer el marco conceptual que fundamenta la pertinencia pedagógica de la propuesta desarrollada en esta investigación.

1.1.1. Antecedentes del aprendizaje mediante el juego

La utilización del juego en la educación posee una sólida tradición teórica. El juego se reconoce como un medio natural para el desarrollo cognitivo, pues permite interactuar con reglas, roles y condiciones simbólicas que estimulan la construcción activa de estructuras mentales. Esta perspectiva se complementa con el enfoque sociocultural, según el cual el juego posibilita niveles superiores de desempeño cognitivo y social mediante la mediación y la interacción significativa. En el campo educativo contemporáneo, estos aportes han evolucionado hacia enfoques que consideran el juego no solo como actividad recreativa, sino como un entorno estructurado donde el aprendizaje ocurre de manera contextualizada, motivante y autónoma (Plass;

Homer y Kinzer, 2020).

Conceptos fundamentales

Para la comprensión de la presente investigación se definen los conceptos fundamentales que enmarcan el objeto de estudio:

Videojuego: Un videojuego es un sistema digital interactivo que integra mecánicas de juego (reglas y objetivos), narrativa (historia o contexto) y tecnología (gráficos, sonido, inteligencia artificial) para generar experiencias inmersivas (Rogers, 2020). Su diseño se fundamenta en principios como la interactividad, los objetivos estructurados y la retroalimentación, que guían el comportamiento del usuario (Johnson y K. Lee, 2021). Además, la adaptabilidad, mediante dificultad ajustable o algoritmos de aprendizaje automático, permite personalizar la experiencia según el estilo de juego individual (Chen; Smith y Alvarez, 2022).

Sostenibilidad: principio rector que busca armonizar el progreso humano con la preservación del medio ambiente y la equidad social, asegurando que las acciones actuales no comprometan el bienestar de las generaciones futuras (Comisión Mundial sobre el Medio Ambiente y el Desarrollo, 1987). La sostenibilidad implica "un desarrollo que equilibra las dimensiones económica, social y ambiental, garantizando que los recursos naturales se utilicen de manera responsable" (Programa de las Naciones Unidas para el Desarrollo, 2021).

Objetivos de Desarrollo Sostenible (ODS): conjunto de 17 metas globales adoptadas por las Naciones Unidas en 2015 como parte de la Agenda 2030 para el Desarrollo Sostenible. Estos objetivos buscan abordar los desafíos más urgentes que enfrenta la humanidad, como la pobreza, la desigualdad, el cambio climático, la degradación ambiental y la justicia (Naciones Unidas, 2015).

Educación para el Desarrollo Sostenible (EDS): enfoque educativo integral que busca empoderar a las personas, comunidades y sociedades para tomar decisiones informadas y responsables que contribuyan a un futuro sostenible. La EDS es "un proceso de aprendizaje que permite a las personas adquirir los conocimientos, las competencias, las actitudes y los valores necesarios para construir un futuro sostenible" (UNESCO, 2020).

Aprendizaje Basado en Juegos (ABJ): metodología educativa que utiliza juegos, ya sean digitales o analógicos, como herramientas pedagógicas para facilitar la adquisición de conocimientos, habilidades y actitudes. El ABJ "se fundamenta en el diseño de entornos lúdicos que estimulan la motivación intrínseca, la exploración autónoma y la resolución de problemas en contextos complejos, replicando dinámicas del mundo real" (Plass; Homer y Kinzer, 2020).

1.1.2. Fundamentos del Aprendizaje Basado en Juegos (Game-Based Learning)

El Aprendizaje Basado en Juegos (GBL, del inglés *Game-Based Learning*) se define como la integración sistemática de mecánicas, reglas y dinámicas propias de los juegos dentro del proceso de enseñanza (Plass; Mayer y Homer, 2020). Los entornos lúdicos proporcionan condiciones particularmente fértiles para la adquisición de conocimientos y el desarrollo de habilidades cognitivas superiores, al permitir la interacción

significativa con los contenidos, la toma de decisiones dentro de escenarios estructurados, la retroalimentación inmediata y el aprendizaje situado en contextos simulados. Estos elementos convierten a los juegos analógicos o digitales en plataformas eficaces para introducir conceptos complejos como los Objetivos de Desarrollo Sostenible, la acción climática o la gestión responsable de recursos.

Estudios recientes demuestran que la aplicación del ABJ en entornos educativos formales mejora significativamente el rendimiento académico y la capacidad de transferir conocimientos a situaciones prácticas (Plass; Homer y Kinzer, 2020, p. 15). Dicha metodología integra elementos como la narrativa inmersiva, la retroalimentación adaptativa y la colaboración en tiempo real, mecanismos que incrementan la participación activa y la retención de conceptos (Deterding; Zagal y Björk, 2020).

Evidencias empíricas del ABJ en sostenibilidad

En el ámbito de la educación para el desarrollo sostenible, videojuegos como *Eco* (simulación de ecosistemas) y *Fate of the World* (políticas climáticas globales) demuestran cómo las herramientas digitales transforman información abstracta en experiencias concretas (Wu y J. J. Lee, 2020). *Eco* es un juego multijugador que simula la construcción de una civilización sostenible en un ecosistema virtual, donde los jugadores deben gestionar recursos naturales, negociar políticas colectivas y enfrentar consecuencias ambientales como la extinción de especies por sobreexplotación. *Fate of the World* desafía a los jugadores a implementar políticas climáticas globales basadas en datos científicos, y ha demostrado mejorar la conciencia sobre la complejidad de la gobernanza ambiental (Flood; Cradock-Henry y Blackett, 2024).

Títulos como *Cities: Skylines* (utilizado en aulas para enseñar planificación urbana sostenible) y *Frostpunk* (que explora dilemas éticos en crisis climáticas) amplían el aprendizaje experiencial mediante mecánicas que vinculan acciones individuales con impactos sistémicos (Hawlitschek; Joeckel y Wittstock, 2022). Estas experiencias demuestran que los videojuegos educativos no solo entretienen, sino que desarrollan habilidades clave para los ODS: pensamiento crítico, colaboración y toma de decisiones basada en evidencia.

1.1.3. Gamificación como estrategia educativa

La gamificación constituye un enfoque complementario al ABJ, centrado en la incorporación de elementos característicos de los juegos —puntos, retroalimentación inmediata, progresión, niveles y desafíos— dentro de situaciones originalmente no lúdicas (Deterding; Zagal y Björk, 2020). En el ámbito educativo, esta estrategia se asocia a incrementos en la motivación intrínseca, la participación activa del estudiantado, la claridad en las metas del aprendizaje y la percepción de logro y autonomía. El uso de mecánicas gamificadas resulta especialmente relevante en recursos formativos vinculados con la sostenibilidad, donde la motivación y el compromiso con la acción son tan importantes como el conocimiento conceptual.

El Consejo Europeo (European Council, 2022) ha reconocido el potencial de la educación digital gamificada como componente central de los planes de acción educativa, promoviendo su integración en todos los niveles formativos de los países miembros. Esta validación institucional a nivel internacional refuerza la pertinencia pedagógica de la presente investigación.

Teorías de aprendizaje aplicadas al diseño de videojuegos educativos

El diseño y análisis de videojuegos con fines educativos requiere integrar modelos que articulen los aspectos pedagógicos, tecnológicos y disciplinares. Dos marcos metodológicos resultan especialmente relevantes para el presente trabajo:

La **Teoría del Flujo** explica cómo los entornos lúdicos mantienen la motivación cuando existe equilibrio entre el desafío y la habilidad del jugador. Esta teoría orienta la organización del ritmo, la dificultad y la retroalimentación dentro del juego educativo.

El **Modelo TPACK** (Tecnológico-Pedagógico-Disciplinar) proporciona un marco para integrar contenido educativo, estrategias pedagógicas y decisiones tecnológicas, permitiendo evaluar la coherencia entre las mecánicas de juego, los objetivos formativos y los recursos digitales seleccionados.

[Figura: Relación entre Teoría del Flujo, TPACK y ABJ]

Figura 1.1. Relación entre los marcos teóricos empleados en el diseño del videojuego educativo (elaboración propia)

1.2. Gamificación en la educación para el desarrollo sostenible

Los videojuegos educativos basados en adaptaciones de juegos de mesa constituyen una categoría especializada que combina la accesibilidad y las mecánicas probadas del formato analógico con las capacidades tecnológicas del medio digital. El presente epígrafe sistematiza los fundamentos teórico-metodológicos asociados a este tipo de desarrollo, específicamente la adaptación digital de juegos de mesa educativos como herramientas para promover los Objetivos de Desarrollo Sostenible, estableciendo los referentes que sustentan la investigación tanto en el contexto internacional como nacional.

1.2.1. Antecedentes internacionales

Desde una perspectiva histórica, los juegos de mesa han constituido herramientas pedagógicas por excelencia durante siglos, ofreciendo entornos estructurados donde los participantes desarrollan habilidades cognitivas, sociales y estratégicas de manera lúdica. La digitalización de estos juegos representa una evolución natural que mantiene la esencia pedagógica del formato original mientras amplifica sus posibilidades mediante la interactividad digital, la retroalimentación inmediata y la eliminación de barreras logísticas (Martínez; Gimenes y Lambert, 2023).

En el ámbito internacional, las Naciones Unidas desarrollaron el juego de mesa *Go Goals!* como herramienta pedagógica oficial para introducir los diecisiete Objetivos de Desarrollo Sostenible a niños de entre ocho y diez años (Naciones Unidas, 2020). Este juego utiliza una estructura inspirada en el clásico Serpientes y Escaleras, adaptando sus mecánicas tradicionales para incorporar contenidos educativos específicos sobre sostenibilidad mediante un sistema de preguntas y respuestas que refuerzan el aprendizaje de cada objetivo.

El juego de mesa Go Goals! como referente pedagógico

La estructura pedagógica de *Go Goals!* se fundamenta en principios de gamificación aplicados a contextos educativos (ibíd.). El juego emplea un tablero de 63 casillas hexagonales que representan el recorrido temporal desde el presente hasta el año 2030, horizonte establecido por la Agenda 2030. Esta metáfora visual del progreso temporal refuerza la comprensión de que la sostenibilidad es un proceso continuo que requiere acciones sostenidas en el tiempo. El tablero incorpora diecisiete casillas temáticas, una por cada ODS, donde los jugadores deben responder preguntas específicas para avanzar.

[Figura: Tablero del juego de mesa Go Goals! (Naciones Unidas, 2020)]

Figura 1.2. Tablero oficial del juego de mesa Go Goals! de las Naciones Unidas (Naciones Unidas, 2020)

Tabla 1.1. Ejemplos de preguntas por nivel cognitivo según ODS (elaboración propia)

ODS	**Pregunta nivel bajo (comprensión)**	**Pregunta nivel alto (aplicación / análisis)**
1. Fin de la pobreza	¿Cómo definir la pobreza? → No puede costear necesidades básicas	¿La pobreza mundial ha disminuido en 25 años? → Sí, 1000 millones han salido de la pobreza
6. Agua limpia y saneamiento	¿Qué es el agua potable? → Agua segura de beber	¿Cómo ahorrar agua en casa? → Duchándote en vez de bañarte
13. Acción por el clima	¿Qué es energía renovable? → Energía de recursos que se renuevan rápido	¿Por qué proteger los bosques? → Producen oxígeno

1.2.2. Antecedentes nacionales

A nivel nacional, aunque el desarrollo de videojuegos educativos orientados a los ODS se encuentra en fase incipiente, existen antecedentes valiosos en programas como Aula Digital y diversas iniciativas universitarias. Según el Informe Nacional Voluntario de Cuba (Grupo Nacional para la Implementación de la Agenda 2030, 2021), se han realizado esfuerzos para mejorar la formación docente y los recursos educativos; sin embargo, persisten desafíos en infraestructura educativa y acceso a tecnologías actualizadas.

Cuba mantiene uno de los sistemas educativos más inclusivos de América Latina. El 99,7 % de su población adulta está alfabetizada (UNESCO, 2023), y la educación es gratuita en todos los niveles tal como lo establece la Constitución de la República (República de Cuba, 2019). No obstante, la escasez de materiales didácticos digitales y las limitaciones de conectividad afectan la implementación de recursos tecnológicos educativos en contextos de crisis económica (Banco Mundial, 2023; Mesa-Lago, 2022).

Brecha identificada en el contexto nacional

La adaptación digital específica de juegos de mesa reconocidos internacionalmente como *Go Goals!* representa un vacío identificado en la oferta actual de recursos educativos tecnológicos, especialmente en la Universidad de las Ciencias Informáticas. No se ha encontrado ningún videojuego desarrollado en Cuba que se centre exclusivamente en la enseñanza de los ODS mediante la mecánica de juego de mesa digital, añadiéndole a esto la pasión que sienten muchas personas por los videojuegos y la gran capacidad que poseen para el Aprendizaje Basado en Juegos.

[Figura: Mapa de recursos educativos digitales sobre ODS en Cuba (elaboración propia)]

Figura 1.3. Estado actual de los recursos educativos digitales sobre ODS en Cuba (elaboración propia)

1.2.3. Estudio de soluciones homólogas

Para profundizar en las alternativas existentes y demostrar la viabilidad y necesidad de la solución propuesta, se analizaron cinco videojuegos que comparten características homólogas con el presente proyecto. La selección, detallada en la Tabla 1.2, incluye títulos que integran mecánicas de juegos de mesa, trivias educativas o enfoques directos en la concienciación sobre la sostenibilidad y los Objetivos de Desarrollo Sostenible.

Como se puede apreciar en dicho análisis, mientras que soluciones comerciales como *Trivia Crack* o *Mario Party* poseen excelentes mecánicas lúdicas de tablero (ver Figura 1.4), carecen de contenido educativo sobre sostenibilidad. Por otro lado, juegos como *World Rescue* se enfocan en los ODS pero mediante minijuegos desconectados, perdiendo la cohesión metafórica de avanzar hacia el 2030 presente en el tablero original de Naciones Unidas. Esto reafirma la brecha existente: la necesidad de un recurso que fusione orgánicamente la mecánica de juego de tablero sobre sostenibilidad (*Go Goals!*) en una plataforma digital completa, dotada de la interactividad que demanda el público infantil actual.

Tabla 1.2. Videojuegos educativos y de tablero relacionados a la sostenibilidad (elaboración propia)

Videojuego	**Temática / Género**	**Conexión a los ODS / Lúdica**
World Rescue	Minijuegos interactivos de resolución de problemas basados en Kenia, Noruega, Brasil, India, etc.	Directa: Oficial de las Naciones Unidas. Los jugadores resuelven problemas como desplazamiento, enfermedad y deforestación (Múltiples ODS).
Trivia Crack (Preguntados)	Juego de preguntas y respuestas competitivo en formato digital.	Referencia mecánica: Utiliza un formato de ruleta y categorías para evaluar conocimiento mixto, inspirando el sistema de evaluación del proyecto.
Mario Party (Serie)	Videojuego de mesa (*party game*) con minijuegos y progreso condicionado por tablero.	Referencia lúdica: Pionero en adaptar la experiencia social y el progreso por tablero analógico a un entorno virtual dinámico e interactivo.
Alba: A Wildlife Adventure	Aventura de exploración centrada en la fotografía y limpieza del entorno natural.	ODS 14 y 15: Promueve fuertemente el activismo ecológico, la restauración de hábitats locales y la preservación de la fauna.
Bleentoro	Rompecabezas lógicos y simulación económica / optimización de recursos.	ODS 9 y 12: Aborda la creación de infraestructuras limpias, gestión de desechos y la producción sostenible sin elementos violentos.

1.3. Análisis del estado actual del juego de mesa Go Goals!

El presente epígrafe se dedica a la descripción y análisis exhaustivo del estado actual del objeto de estudio: el juego de mesa *Go Goals!* en su formato físico original. Este análisis demuestra la pertinencia de la investigación y valida tanto la situación problemática como el problema científico planteado.

[Figura: Interfaz típica de un tablero digital competitivo en la serie Mario Party]

Figura 1.4. Mecánica de tablero digital interactivo en la serie comercial *Mario Party (elaboración propia)*

1.3.1. Descripción del juego Go Goals! en su formato original

El juego de mesa *Go Goals!* constituye un recurso educativo oficial desarrollado por las Naciones Unidas con el propósito específico de introducir los diecisiete Objetivos de Desarrollo Sostenible a poblaciones infantiles de manera lúdica y accesible (Naciones Unidas, 2020). Dirigido específicamente a niños de ocho a diez años, el juego busca promover el aprendizaje activo, la interacción social y la motivación por la acción sostenible mediante mecánicas de juego probadas durante décadas. Su estructura se basa en el clásico juego Serpientes y Escaleras, adaptado para incorporar contenidos educativos específicos sobre cada uno de los ODS mediante un sistema integrado de preguntas y respuestas.

Tabla 1.3. Componentes del juego de mesa Go Goals! (elaboración propia)

Componente	**Descripción**
Tablero	63 casillas hexagonales conectadas en un recorrido no lineal. Incluye 17 casillas temáticas (una por ODS), escaleras (avance positivo) y serpientes (retroceso correctivo).
Fichas de jugadores	6 fichas personalizables con el personaje Elyx (mascota oficial de la ONU).
Dado	Un dado de 6 caras para determinar el avance aleatorio.
Cartas de preguntas	34 cartas (2 por ODS), con preguntas de opción múltiple, verdadero/falso y selección múltiple.
Instrucciones	Reglas completas en español, inglés y otros idiomas.

Mecánica pedagógica del juego

La mecánica pedagógica de *Go Goals!* combina el azar del movimiento por el tablero con la evaluación del conocimiento sobre los ODS. Cuando un jugador cae en una casilla temática, debe responder correctamente una pregunta para mantenerse en su posición o avanzar, mientras que una respuesta incorrecta puede provocar un retroceso mediante el efecto de las serpientes. Este diseño asegura que el aprendizaje esté integrado en la dinámica del juego, no como una actividad separada, sino como el mecanismo central de

progreso (Naciones Unidas, 2020).

1.3.2. Diagnóstico: fortalezas y limitaciones del formato físico

Un análisis riguroso del juego en su formato físico actual revela tanto fortalezas significativas como limitaciones críticas que justifican la necesidad de una adaptación digital. Las fortalezas incluyen un alto valor pedagógico con mecánicas probadas, lenguaje accesible para el grupo de edad objetivo y una progresión cognitiva según la taxonomía de Bloom. Entre las limitaciones más relevantes se destacan el contenido estático derivado de solo 34 preguntas fijas, la dependencia de recursos de impresión a color, la imposibilidad de uso en entornos de educación a distancia y la ausencia de mecanismos de registro del aprendizaje individual.

Tabla 1.4. Diagnóstico del estado actual de Go Goals! (formato físico) (elaboración propia)

Aspecto	**Fortalezas**	**Limitaciones**
Diseño pedagógico	Alta alineación con ODS, mecánicas probadas, lenguaje accesible para 8-10 años.	Contenido estático: solo 34 preguntas fijas. Rejugabilidad limitada por memorización.
Interacción social	Fomenta cooperación, genera discusión sobre sostenibilidad, enseña respeto de turnos.	Requiere grupo físico y materiales impresos. Imposible en educación a distancia.
Accesibilidad	Gratuito, multilingüe, formato imprimible disponible en portal oficial de la ONU.	Dependiente de impresora a color; inaccesible en zonas rurales o de bajos recursos.
Escalabilidad	Ideal para aulas pequeñas (4-6 jugadores) con recursos adecuados.	Imposible jugar en línea o con grupos grandes. No permite distribución masiva.
Medición del aprendizaje	Retroalimentación cualitativa inmediata (respuesta correcta → avance).	Sin registro de progreso longitudinal ni evaluación cuantitativa.

Justificación de la adaptación digital

La adaptación digital propuesta mediante Godot Engine resuelve las limitaciones identificadas al ofrecer: acceso universal mediante descarga gratuita en múltiples plataformas, un banco de preguntas ampliable mediante archivos de datos estructurados (JSON), registro automático del progreso y patrones de aprendizaje, y una experiencia enriquecida con animaciones, sonido y efectos visuales. La situación problemática identificada puede caracterizarse como un recurso educativo de alto valor pedagógico cuyo alcance e impacto potencial se ven severamente limitados por restricciones inherentes a su formato analógico.

1.4. Metodologías y herramientas para el desarrollo de Go Goals! Digital

El presente epígrafe se dedica a la sistematización de los fundamentos teórico-metodológicos asociados a las tecnologías y herramientas utilizadas para lograr el resultado contenido en el campo de acción. Se analizan y justifican la metodología de desarrollo, el motor de videojuegos, los lenguajes de programación y modelado, así como las herramientas de soporte seleccionadas. La elección de estas tecnologías se basó en su capacidad para facilitar un desarrollo ágil, eficiente y alineado con los ODS, así como en su adaptabilidad a un entorno académico con recursos y tiempo limitados.

1.4.1. Metodología de desarrollo de software

Una metodología de desarrollo de software es un conjunto de principios, prácticas y procesos sistemáticos que guían la planificación, diseño, implementación, prueba y mantenimiento de sistemas de software. Las metodologías de desarrollo "proporcionan un marco estructurado para gestionar la complejidad de los proyectos de software, asegurando que se cumplan los requisitos del cliente y se entregue un producto de alta calidad dentro del tiempo y el presupuesto establecidos" (Sommerville, 2021). Estas metodologías pueden variar desde enfoques tradicionales, como el modelo en cascada, que sigue una secuencia lineal de fases, hasta enfoques ágiles, como Scrum o Extreme Programming (XP), que promueven la flexibilidad, la colaboración y la entrega iterativa de funcionalidades (Schwaber y Sutherland, 2020).

Para el desarrollo del videojuego educativo *Go Goals! Digital* se seleccionó Extreme Programming (XP). Esta metodología ágil, formulada originalmente por Beck en 1999 (Beck, 1999), enfatiza la flexibilidad, la colaboración y la entrega continua de valor al cliente. Se centra en prácticas como la programación en parejas, el desarrollo guiado por pruebas (TDD) y la integración continua (Cockburn, 2020). XP promueve un entorno de trabajo colaborativo donde los desarrolladores y los clientes trabajan juntos para definir las características del software, facilitando una mejor alineación entre las necesidades del usuario y el producto final.

La elección de XP se justifica por la naturaleza dinámica del desarrollo de videojuegos educativos, que requiere adaptarse rápidamente a los cambios en el diseño y las mecánicas del juego (Schwaber y Sutherland, 2020). La colaboración estrecha entre desarrollador y cliente es fundamental para garantizar que el contenido del juego esté alineado con los ODS. El énfasis de XP en la calidad del software, mediante pruebas continuas y refactorización, asegura que *Go Goals! Digital* sea a la vez funcional y un excelente recurso educativo.

1.4.2. Herramienta CASE

Las herramientas CASE (Computer-Aided Software Engineering) son software diseñado para apoyar el desarrollo, mantenimiento y gestión de sistemas de software, automatizando y optimizando diversas etapas del ciclo de vida del mismo. Estas herramientas "facilitan el modelado, diseño, implementación y documentación de sistemas de software, mejorando la productividad y la calidad del producto final" (Pressman

[Figura: Fases de Extreme Programming aplicadas al proyecto (elaboración propia)]

Figura 1.5. Fases de Extreme Programming aplicadas al desarrollo de *Go Goals! Digital (elaboración propia)* (elaboración propia)

y Maxim, 2021). Abarcan una amplia gama de funcionalidades, como la creación de diagramas con UML, la gestión de requisitos, la generación de código y la documentación automatizada (Sommerville, 2021).

Se utilizó Visual Paradigm en su versión 8.0 como herramienta CASE para apoyar el diseño, modelado y documentación del sistema. Visual Paradigm es una plataforma integral que ofrece un conjunto de herramientas para el análisis y diseño de software, incluyendo diagramas UML, gestión de requisitos, modelado de bases de datos y prototipado de interfaces de usuario (Visual Paradigm, 2023). Su versatilidad y facilidad de uso la convierten en una opción ideal para proyectos de ingeniería de software, especialmente en entornos académicos.

Visual Paradigm permite a los desarrolladores crear y gestionar modelos de software de manera eficiente. Sus características principales incluyen soporte de diagramas UML (clases, secuencia, casos de uso y actividades), gestión y rastreo de requisitos del sistema, prototipado de interfaces que facilita el diseño de videojuegos, y funciones de colaboración en línea (ibíd.).

1.4.3. Lenguaje de modelado

El lenguaje de modelado utilizado es UML (Unified Modeling Language) en su versión 2.5.1. UML es un lenguaje de modelado visual estandarizado que se utiliza para representar, diseñar y documentar sistemas de software. En su esencia, UML constituye "una herramienta esencial para visualizar la estructura, el comportamiento y las interacciones de un sistema, permitiendo a los equipos de desarrollo comunicar ideas complejas de manera clara y precisa" (Fowler, 2018). Se utiliza como herramienta fundamental para modelar los diferentes aspectos del sistema de *Go Goals! Digital*, desde la arquitectura general hasta los detalles de implementación.

UML aporta ventajas clave al proyecto: proporciona comunicación efectiva al constituir un lenguaje común para desarrolladores y los tutores del proyecto; ofrece documentación clara al servir de representación visual que facilita la evolución del software; y permite la detección temprana de errores al modelar el sistema antes de la implementación, posibilitando la identificación y corrección de problemas de diseño en etapas tempranas (ibíd.).

1.4.4. Motor de videojuegos

Un motor de videojuegos es un conjunto de herramientas y tecnologías que permiten el desarrollo, la creación y la ejecución de videojuegos, proporcionando una infraestructura que incluye gráficos, física, sonido, inteligencia artificial y otras funcionalidades necesarias para crear un entorno interactivo y dinámico (Gregory, 2020). Para el desarrollo del videojuego educativo se seleccionó Godot Engine en su versión 4.4.1 (Godot Engine, 2023).

Godot es un motor de código abierto bajo licencia MIT que ha ganado considerable popularidad en la comunidad de desarrollo de videojuegos por su flexibilidad, accesibilidad y facilidad de uso (Linietsky y Manzur, 2020). En su versión 4.4.1, el motor presenta mejoras significativas en el rendimiento gráfico, soporte robusto para gráficos bidimensionales optimizados y un sistema de scripting mejorado basado en GDScript. Las capacidades gráficas avanzadas de Godot 4.4.1 son ideales para crear un videojuego educativo atractivo que capte la atención de los estudiantes. La comunidad activa y el amplio soporte documental de Godot facilitan el aprendizaje y la resolución de problemas durante el proceso de desarrollo (Godot Engine, 2023).

Tabla 1.5. Comparativa ampliada de motores de videojuegos según criterios del proyecto (elaboración propia)

Motor	**Licencia y Costos**	**Soporte 2D y Rendimiento**	**Exportación Multiplataforma**	**Curva de Aprendizaje**
Godot (v4.4.1)	Código abierto (MIT). Totalmente gratuito, sin regalías ni costos de suscripción.	Motor 2D dedicado real (basado en píxeles). Alta optimización para equipos de bajos recursos.	Soporte nativo y eficiente para Windows, Linux y macOS sin necesidad de complementos.	Baja. Uso de GDScript (sintaxis inspirada en Python), lo que agiliza el desarrollo de prototipos.
Unity	Propietaria. Gratuito hasta cierto nivel de ingresos; sujeto a inestabilidad por cambios de políticas.	Soporte 2D robusto, pero integrado sobre un núcleo 3D, lo que añade sobrecarga innecesaria al proyecto.	Excelente soporte multiplataforma, aunque genera ejecutables considerablemente más pesados.	Media-alta. Requiere dominio de C# y navegar una interfaz de editor considerablemente más densa.
Unreal Engine	Propietaria. Gratuito inicialmente, pero exige pago de regalías tras superar límites de ventas.	Orientado mayormente a gráficos 3D fotorrealistas de alta fidelidad. Subutilizado en juegos 2D simples.	Soporta las principales plataformas de escritorio, pero con requerimientos mínimos de hardware muy elevados.	Alta. Basado en C++ o el sistema visual Blueprints, con altos requerimientos de hardware para ejecución.
Construct 3	Propietaria. Modelo de negocio basado en suscripción periódica de pago obligatorio.	Excelente herramienta específica para desarrollo 2D rápido y sin requerir gran potencia computacional.	Aunque permite exportación a escritorio, está concebida de forma nativa para entornos web (HTML5).	Muy baja. Basado puramente en un entorno visual y hojas de eventos, sin programación de código tradicional.

[Figura: Comparativa de motores de videojuegos disponibles (elaboración propia)]

Figura 1.6. Comparativa de motores de videojuegos por criterios del proyecto (elaboración propia)

1.4.5. Lenguaje de programación

Un lenguaje de programación es un sistema formal de comunicación que permite a los desarrolladores escribir instrucciones precisas para que una computadora ejecute tareas específicas. Los lenguajes de programación son herramientas esenciales en el desarrollo de software, ya que permiten crear aplicaciones, sistemas operativos, videojuegos y otras soluciones tecnológicas, y evolucionan constantemente para adaptarse a las necesidades emergentes (Sommerville, 2021).

Se utilizó GDScript como lenguaje de programación principal. Este es un lenguaje de scripting dinámico y de alto nivel diseñado específicamente para el motor de videojuegos Godot Engine. Su sintaxis es similar a Python, lo que lo hace accesible para programadores de todos los niveles (Godot Engine Documentation, 2023). GDScript está optimizado para la creación de videojuegos, ofreciendo una integración nativa con las funcionalidades de Godot como la gestión de escenas, nodos y recursos, lo que permite un desarrollo ágil y eficiente (Linietsky y Manzur, 2020).

La elección de GDScript para *Go Goals! Digital* se justifica por su compatibilidad nativa con Godot Engine, la rapidez en el desarrollo, y su simplicidad que permite implementar mecánicas educativas complejas —como el sistema de preguntas por ODS, la lógica de escaleras y serpientes, y la gestión de turnos— en menos tiempo comparado con otros lenguajes como C# o C++. Su facilidad para la creación de prototipos rápidos y la iteración constante es un factor esencial en el proceso de desarrollo bajo la metodología XP. Además, la comunidad activa y los recursos documentales disponibles permiten un desarrollo más eficiente (Godot Engine Documentation, 2023).

1.4.6. Herramienta de diseño de activos gráficos

El diseño de los activos gráficos bidimensionales del videojuego, incluyendo el tablero hexagonal, los íconos de los ODS y las fichas de los jugadores, requiere una herramienta de edición vectorial capaz de producir gráficos escalables y de alta calidad. Se seleccionó Inkscape en su versión 1.3 como herramienta principal para el diseño del tablero y los elementos gráficos vectoriales del juego.

Inkscape es un editor de gráficos vectoriales de código abierto y gratuito, compatible con el estándar SVG (Scalable Vector Graphics), que permite crear ilustraciones, íconos y diagramas de calidad profesional

(Godot Engine Documentation, 2023). Su elección se justifica por su naturaleza gratuita y de código abierto, su capacidad de exportar en formatos compatibles con Godot (PNG, SVG), y la posibilidad de ajustar fácilmente los colores oficiales de cada ODS (definidos por las Naciones Unidas) para cada casilla temática del tablero.

El contenido educativo del videojuego —el banco de preguntas por ODS— se gestiona mediante archivos estructurados en formato JSON, lo que permite a educadores sin conocimientos de programación añadir, modificar o traducir preguntas mediante cualquier editor de texto (ibíd.).

[Figura: Diseño del tablero hexagonal en Inkscape (elaboración propia)]

Figura 1.7. Diseño vectorial del tablero hexagonal de *Go Goals! Digital (elaboración propia)* en Inkscape (elaboración propia)

1.4.7. Sistema de control de versiones

El sistema de control de versiones utilizado en el desarrollo de la propuesta de solución es Git en su versión 2.43.0. Git es un sistema de control de versiones distribuido ampliamente utilizado para gestionar cambios en el código fuente durante el desarrollo de software. Según Chacon y Straub (2014), Git permite a los desarrolladores "rastrear modificaciones en archivos, coordinar el trabajo en equipo y mantener un historial completo de cambios, facilitando la colaboración y la reversión a versiones anteriores en caso de errores". A diferencia de los sistemas centralizados, Git opera de manera distribuida, lo que significa que cada desarrollador tiene una copia completa del repositorio, incluyendo su historial de cambios (ibíd.).

Git utiliza un modelo basado en *snapshots* (instantáneas) en lugar de diferencias entre archivos, lo que lo hace eficiente en términos de almacenamiento y rendimiento. Ofrece funcionalidades avanzadas como ramificaciones (*branching*) y fusiones (*merging*), permitiendo trabajar en múltiples características de manera simultánea sin interferencias (ibíd.). Para *Go Goals! Digital* esto resulta esencial para gestionar las iteraciones de XP de manera organizada y revertir cambios ante errores en el desarrollo de las mecánicas de juego.

1.4.8. Software de gestión de proyectos

Se establece GitHub Projects como software de gestión de proyectos. Esta es una herramienta integrada en GitHub que permite a los equipos de desarrollo gestionar y organizar tareas, *issues* y *pull requests*

de manera eficiente (GitHub, Inc., 2022). GitHub Projects es una solución flexible para la planificación y seguimiento del progreso de proyectos de software, ofreciendo una vista visual del flujo de trabajo y las prioridades del equipo.

Esta herramienta es especialmente útil para equipos que siguen metodologías ágiles como XP, ya que permite crear tableros personalizados con columnas que representan diferentes etapas del desarrollo: "Por hacer", "En progreso" y "Terminado". GitHub Projects incluye características como la automatización de tareas, que mueve automáticamente las tareas entre columnas según criterios definidos, y la integración con *issues* y *pull requests*, lo que facilita el seguimiento del progreso y la trazabilidad de las tareas (ibíd.).

[Figura: Tablero de GitHub Projects del proyecto (elaboración propia)]

Figura 1.8. Tablero de gestión de tareas en GitHub Projects para *Go Goals! Digital (elaboración propia)* (elaboración propia)

El conjunto de herramientas seleccionadas para el desarrollo de *Go Goals! Digital* forma un ecosistema tecnológico coherente y complementario. La metodología XP establece el marco de trabajo iterativo; Visual Paradigm y UML garantizan el modelado y la documentación del sistema; Godot Engine 4.4.1 y GDScript constituyen el núcleo de la implementación; Inkscape aporta los activos gráficos vectoriales del tablero; Git asegura la trazabilidad de cambios; y GitHub Projects canaliza la gestión de tareas bajo el enfoque ágil. La integración de estas herramientas permite un flujo de trabajo continuo, donde cada decisión tecnológica refuerza a las demás, reduciendo la deuda técnica y garantizando la mantenibilidad del producto final.

Conclusiones del capítulo

A partir de los contenidos abordados en el presente capítulo, se establecen las siguientes conclusiones:

1. Los referentes teóricos sobre Aprendizaje Basado en Juegos, gamificación, Teoría del Flujo y Modelo TPACK confirman la pertinencia pedagógica de los videojuegos educativos como mediadores para la enseñanza de los Objetivos de Desarrollo Sostenible.
2. El diagnóstico del estado actual del juego de mesa *Go Goals!* evidenció que, si bien posee un diseño pedagógico sólido y alineado con los ODS, su formato físico presenta limitaciones estructurales que afectan su accesibilidad, escalabilidad, rejugabilidad y capacidad para registrar el aprendizaje, lo cual valida plenamente el problema científico planteado.

3. La selección de Godot Engine 4.4.1 como motor de desarrollo, sustentada en el análisis comparativo con alternativas tecnológicas, resulta la opción que mejor cumple los criterios de accesibilidad universal, código abierto, capacidad gráfica 2D y bajo consumo de recursos requeridos por el proyecto.
4. La metodología Extreme Programming se justifica como el enfoque de desarrollo más adecuado para el presente proyecto por su capacidad de adaptación iterativa, la colaboración estrecha con usuarios educativos y el énfasis en la calidad del software mediante pruebas continuas.
5. Se seleccionaron y justificaron las herramientas tecnológicas que conforman el entorno de desarrollo: Visual Paradigm como herramienta CASE, UML como lenguaje de modelado, GDScript como lenguaje de programación, Inkscape como herramienta de diseño gráfico, Git como sistema de control de versiones y GitHub Projects como software de gestión del proyecto.

CAPÍTULO 2

Planificación, diseño e implementación de la solución propuesta

El presente capítulo tiene como objetivo presentar el proceso de análisis, diseño e implementación del videojuego educativo *Go Goals! Digital*, detallando las decisiones técnicas y metodológicas que sustentan su desarrollo. Este se estructura en la descripción de la propuesta de solución y tres ejes principales guiados por la metodología XP: (1) planificación, donde se definen los requisitos y la organización del proyecto; (2) diseño, que abarca la arquitectura del sistema, los prototipos de interfaces y las tarjetas CRC; y (3) implementación, que describe los estándares de codificación, los patrones de diseño y las tareas de ingeniería. A través de este análisis, se busca proporcionar una visión clara y sistemática de cómo se materializó el proyecto, desde la obtención de requisitos hasta la implementación final, respaldado por tablas, modelos y diagramas que garantizan la calidad del producto.

2.1. Descripción de la propuesta de solución

Go Goals! Digital es un videojuego educativo basado en la adaptación digital del juego de mesa oficial *Go Goals!* de las Naciones Unidas (Naciones Unidas, 2020). En su formato digital, los jugadores (de 1 a 6) controlan fichas en un tablero hexagonal de 63 casillas que metaforiza el recorrido colectivo hacia el año 2030, horizonte establecido por la Agenda 2030 para el Desarrollo Sostenible (Naciones Unidas, 2015). El avance se determina mediante un dado virtual, y al caer en una de las 17 casillas temáticas (una por ODS), el jugador debe responder una pregunta de opción múltiple sobre el objetivo correspondiente. Las respuestas correctas activan el efecto de escalera (avance positivo), mientras que las incorrectas activan el efecto de serpiente (retroceso correctivo), preservando la mecánica pedagógica central del juego original.

Las principales mejoras que introduce la versión digital respecto al formato físico son:

- **Accesibilidad universal**: exportación nativa a plataformas de escritorio (Windows, Linux y macOS), sin necesidad de materiales físicos ni impresión.
- **Contenido dinámico**: banco de preguntas ampliable en formato JSON, editable por educadores sin conocimientos de programación.

- **Retroalimentación enriquecida**: animaciones de fichas, efectos de sonido, feedback visual inmediato (colores, partículas) al responder.
- **Seguimiento del aprendizaje**: registro automático de respuestas correctas e incorrectas por jugador y por ODS.
- **Rejugabilidad**: selección aleatoria y sin repetición de preguntas dentro de cada sesión, evitando la memorización.

[Figura: Vista general del videojuego Go Goals! Digital (elaboración propia)]

Figura 2.1. Vista general del videojuego educativo *Go Goals! Digital (elaboración propia)* (elaboración propia)

Dentro del ecosistema de recursos educativos digitales sobre desarrollo sostenible, *Go Goals! Digital* ocupa un nicho específico: es el único videojuego desarrollado en Cuba que adapta fielmente la mecánica del juego de mesa oficial de las Naciones Unidas a un entorno digital interactivo. A diferencia de otras iniciativas educativas basadas en vídeos o materiales estáticos, este videojuego integra la competencia entre varios jugadores, el azar controlado y la evaluación del conocimiento en una sola experiencia lúdica. La plataforma Godot Engine, elegida por su licencia libre (MIT) y su capacidad de exportación multiplataforma, garantiza que el juego pueda ejecutarse de manera nativa en diversos sistemas operativos de escritorio (Windows, Linux y macOS), sin requerir configuraciones complejas adicionalmente, lo que es especialmente relevante en contextos educativos con recursos tecnológicos limitados (Godot Engine, 2023).

El público objetivo son niños y jóvenes de entre 8 y 14 años, con especial énfasis en el contexto escolar cubano, donde la disponibilidad de materiales didácticos digitales actualizados sobre los ODS es aún escasa (Grupo Nacional para la Implementación de la Agenda 2030, 2021). El diseño del videojuego respeta los colores, íconos y denominaciones oficiales de cada ODS definidos por las Naciones Unidas, lo que refuerza la identificación visual del contenido educativo y facilita su uso como complemento de programas curriculares relacionados con la sostenibilidad.

2.2. Planificación

El presente epígrafe se enfoca en la organización y gestión del proyecto siguiendo la metodología Extreme Programming (XP). La planificación en XP se centra en la flexibilidad y la colaboración continua con

el cliente, utilizando iteraciones cortas para adaptarse a cambios y priorizar funcionalidades (Beck, 1999). Aquí se detallan los procesos clave que guían el desarrollo de *Go Goals! Digital*, desde la obtención de requisitos hasta la definición de historias de usuario, la estimación de tiempo y el plan de iteraciones.

Como parte inicial de esta planificación, se realizaron sesiones de tormenta de ideas (brainstorming) para enfocar el proyecto en la arquitectura general y posteriormente reuniones que resultaron en la definición de los requisitos funcionales, no funcionales e historias de usuario (Schwaber y Sutherland, 2020).

2.2.1. Requisitos funcionales

Los requisitos funcionales son descripciones detalladas de las funciones, comportamientos y procesos que un sistema de software debe cumplir para satisfacer las necesidades de sus usuarios y alcanzar los objetivos del negocio. En esencia, definen "lo que el sistema debe hacer" al especificar, de forma clara y verificable, las entradas que recibe, las operaciones que realiza y las salidas que produce. Según Pressman y Maxim (Pressman y Maxim, 2021), esta definición es esencial no solo para orientar las fases de análisis, diseño, implementación y pruebas, sino también para asegurar una comunicación efectiva entre todos los involucrados en el proyecto, evitando ambigüedades y facilitando la gestión de cambios durante el desarrollo.

Tabla 2.1. Requisitos funcionales del videojuego Go Goals! Digital (elaboración propia)

No.	Nombre	Descripción	Prioridad	Complejidad
1.	Inicio de partida	El sistema debe permitir al usuario iniciar una nueva partida configurando el número de jugadores (1-6).	Alta	Baja
2.	Visualización del tablero	El sistema debe mostrar el tablero hexagonal completo con las 63 casillas correctamente identificadas.	Alta	Media
3.	Lanzamiento de dado	El sistema debe simular el lanzamiento de un dado de 6 caras con resultado pseudoaleatorio.	Alta	Baja
4.	Movimiento de ficha	El sistema debe mover la ficha del jugador activo la cantidad de casillas indicada por el dado con animación.	Alta	Media
5.	Efecto de escalera	El sistema debe detectar casillas de escalera y aplicar el avance correspondiente con animación.	Alta	Media
6.	Efecto de serpiente	El sistema debe detectar casillas de serpiente y aplicar el retroceso correspondiente con animación.	Alta	Media
7.	Mostrar pregunta ODS	El sistema debe mostrar una pregunta aleatoria del banco cuando una ficha cae en casilla temática.	Alta	Alta

Tabla 2.1 – Continuación de la página anterior

No.	Nombre	Descripción	Prioridad	Complejidad
8.	Evaluar respuesta	El sistema debe evaluar la respuesta del jugador, mostrar retroalimentación y aplicar el efecto correspondiente.	Alta	Media
9.	Gestión de turnos	El sistema debe gestionar el turno de cada jugador de forma automática y mostrar el jugador activo.	Alta	Media
10.	Condición de victoria	El sistema debe detectar cuando un jugador llega a la casilla 63 y mostrar la pantalla de victoria.	Alta	Baja
11.	Estadísticas de partida	El sistema debe registrar y mostrar las respuestas correctas e incorrectas por jugador y por ODS.	Media	Alta
12.	Menú principal	El sistema debe contar con un menú principal con opciones de iniciar partida, ver instrucciones y salir.	Alta	Baja
13.	Pantalla de instrucciones	El sistema debe mostrar las reglas del juego de manera clara y accesible.	Media	Baja
14.	Configuración de audio	El sistema debe permitir activar o desactivar música y efectos de sonido.	Baja	Baja

2.2.2. Requisitos no funcionales

Los requisitos no funcionales son aquellos que especifican criterios que pueden usarse para juzgar la operación de un sistema, en lugar de sus comportamientos específicos. Estos requisitos se centran en aspectos como el rendimiento, la seguridad, la usabilidad y la fiabilidad del sistema, y son esenciales para garantizar que el software cumpla con las expectativas de calidad y satisfacción del usuario final (Pressman y Maxim, 2021).

Tabla 2.2. Requisitos no funcionales del videojuego Go Goals! Digital (elaboración propia)

Nombre	Tipo	Descripción
Interfaz accesible para niños	Usabilidad	La interfaz gráfica debe emplear tipografía de tamaño no menor a 16pt, botones grandes y contraste alto, adecuados para usuarios de 8 a 10 años.
Tiempo de respuesta	Rendimiento	El sistema debe responder a las acciones del usuario (lanzar dado, seleccionar respuesta) en menos de 0,5 segundos.
Compatibilidad multiplataforma	Compatibilidad	El videojuego debe ser compatible con la última versión estable de los sistemas operativos Windows 10+, Linux y macOS.

Tabla 2.2 – Continuación de la página anterior

Nombre	**Tipo**	**Descripción**
Tutorial contextual	Usabilidad	Al presentarse una funcionalidad nueva, el sistema debe mostrar un tutorial o indicación breve.
Documentación interna del código	Mantenibilidad	El código GDScript debe generar documentación interna mediante comentarios de documentación accesibles para futuras iteraciones.
Disponibilidad offline	Disponibilidad	La versión de escritorio debe funcionar completamente sin conexión a internet.
Tamaño del ejecutable	Rendimiento	El ejecutable para Windows no debe superar 150 MB para facilitar su distribución en contextos con ancho de banda limitado.

2.2.3. Historias de usuario

Las historias de usuario son una técnica fundamental en metodologías ágiles como Extreme Programming (XP), utilizadas para capturar requisitos desde la perspectiva del usuario final. Según Cohn (Cohn, 2020), una historia de usuario es "una descripción breve y simple de una funcionalidad deseada, expresada en lenguaje cotidiano y centrada en el valor que aporta al usuario". A diferencia de los documentos de requisitos tradicionales, las historias de usuario son más flexibles, pero siguen el formato estandarizado:

"Como [rol], quiero [acción], para [beneficio]"

A continuación se presentan las historias de usuario definidas para las funcionalidades clave del videojuego:

Tabla 2.3. Historia de usuario 1: Inicio de nueva partida educativa (elaboración propia)

Historia de Usuario	
Número: 1	**Nombre:** Inicio de nueva partida educativa
Prioridad: Alta	**Riesgo:** Medio
Puntos de estimación: 2.0	**Iteración asignada:** 1
Descripción: Como jugador quiero iniciar una nueva partida para aprender sobre los ODS de manera interactiva.	
Observaciones: • Debe incluir botón para iniciar partida. • Permite configurar número de jugadores (1–6). • Carga el tablero y realiza transición a la primera ronda. • Se inicializan estadísticas en cero.	

Tabla 2.4. Historia de usuario 2: Visualización del tablero hexagonal (elaboración propia)

Historia de Usuario	
Número: 2	**Nombre:** Visualización del tablero hexagonal
Prioridad: Alta	**Riesgo:** Alto
Puntos de estimación: 5.0	**Iteración asignada:** 1
Descripción: Como jugador quiero ver el tablero completo de 63 casillas para entender el recorrido hacia el año 2030.	
Observaciones: • El tablero debe ser fiel al diseño oficial. • Las 17 casillas temáticas deben identificarse con el ícono del ODS correspondiente. • El tablero debe ser visible sin scroll. • Debe ser adaptable a distintas resoluciones de pantalla.	

Tabla 2.5. Historia de usuario 3: Lanzamiento de dado virtual (elaboración propia)

Historia de Usuario	
Número: 3	**Nombre:** Lanzamiento de dado virtual
Prioridad: Alta	**Riesgo:** Medio
Puntos de estimación: 1.5	**Iteración asignada:** 1
Descripción: Como jugador quiero lanzar un dado virtual para determinar el avance de mi ficha en el tablero.	
Observaciones: • Debe mostrar animación breve del dado. • El resultado entre 1 y 6 debe ser pseudoaleatorio. • No debe ser posible lanzar el dado más de una vez por turno. • Debe incluir feedback visual y sonoro al obtener el resultado.	

Tabla 2.6. Historia de usuario 4: Movimiento animado de ficha (elaboración propia)

Historia de Usuario	
Número: 4	**Nombre:** Movimiento animado de ficha
Prioridad: Alta	**Riesgo:** Alto
Puntos de estimación: 4.0	**Iteración asignada:** 1
Descripción: Como jugador quiero que mi ficha se mueva automáticamente según el resultado del dado para concentrarme en el contenido educativo.	
Observaciones: • El movimiento debe ser animado casilla por casilla. • Debe calcularse la trayectoria correcta. • Al llegar a la casilla final se comprueba si es casilla ODS, escalera o serpiente.	

Tabla 2.7. Historia de usuario 5: Efecto de escalera y serpiente (elaboración propia)

Historia de Usuario	
Número: 5	**Nombre:** Efecto de escalera y serpiente
Prioridad: Alta	**Riesgo:** Medio
Puntos de estimación: 3.0	**Iteración asignada:** 2
Descripción: Como jugador quiero que el juego aplique el efecto de escalera o serpiente cuando corresponda para experimentar la mecánica original del juego.	
Observaciones: • El sistema debe detectar la casilla final. • En casilla de escalera la ficha debe avanzar a la casilla destino con animación de subida. • En casilla de serpiente la ficha retrocede con animación descendente. • Debe incluir feedback sonoro diferenciado.	

Tabla 2.8. Historia de usuario 6: Preguntas educativas sobre ODS (elaboración propia)

Historia de Usuario	
Número: 6	**Nombre:** Preguntas educativas sobre ODS
Prioridad: Alta	**Riesgo:** Alto
Puntos de estimación: 5.0	**Iteración asignada:** 2
Descripción: Como jugador quiero responder preguntas cuando caiga en una casilla temática de ODS para aprender interactivamente sobre la sostenibilidad.	
Observaciones: • El juego debe pausar el turno al caer en casilla ODS. • Se debe mostrar un panel modal con el ícono del ODS, el texto de la pregunta y las opciones de respuesta. • El texto debe ser legible para niños de 8–10 años. • Los botones deben ser de tamaño grande. • Se selecciona una pregunta aleatoria sin repetir dentro de la sesión. • Puede activarse límite de tiempo opcional.	

Tabla 2.9. Historia de usuario 7: Evaluación y retroalimentación de respuestas (elaboración propia)

Historia de Usuario	
Número: 7	**Nombre:** Evaluación y retroalimentación de respuestas
Prioridad: Alta	**Riesgo:** Medio
Puntos de estimación: 3.0	**Iteración asignada:** 2
Descripción: Como jugador quiero recibir retroalimentación inmediata al responder para entender si acerté y por qué.	
Observaciones: • El sistema debe mostrar un indicador visual de resultado (verde/rojo). • Debe mostrar un mensaje explicativo breve de la respuesta correcta. • La duración del feedback debe ser de aproximadamente 5 segundos. • Si la respuesta es correcta se mantiene la posición o se avanza por escalera. • Si es incorrecta se aplica el retroceso.	

Tabla 2.10. Historia de usuario 8: Gestión de turnos automática (elaboración propia)

Historia de Usuario	
Número: 8	**Nombre:** Gestión de turnos automática
Prioridad: Alta	**Riesgo:** Baja
Puntos de estimación: 2.0	**Iteración asignada:** 2
Descripción: Como jugador quiero que el sistema gestione los turnos automáticamente para que el juego fluya sin interrupciones.	
Observaciones: • El sistema debe indicar visualmente el jugador activo. • Al finalizar el turno debe pasar automáticamente al siguiente jugador en orden. • En modo de un jugador contra la IA, el turno de la IA debe ejecutarse automáticamente.	

Tabla 2.11. Historia de usuario 9: Condición de victoria (elaboración propia)

Historia de Usuario	
Número: 9	**Nombre:** Condición de victoria
Prioridad: Alta	**Riesgo:** Baja
Puntos de estimación: 1.5	**Iteración asignada:** 3
Descripción: Como jugador quiero que el juego detecte cuando llegue a la casilla 63 para saber que gané.	
Observaciones: • El sistema debe detectar la llegada a la casilla final. • Debe mostrar una pantalla de victoria con animación de confeti y el nombre del ganador. • Debe ofrecer opciones de jugar de nuevo o volver al menú principal.	

Tabla 2.12. Historia de usuario 10: Estadísticas de aprendizaje (elaboración propia)

Historia de Usuario	
Número: 10	**Nombre:** Estadísticas de aprendizaje
Prioridad: Media	**Riesgo:** Medio
Puntos de estimación: 4.0	**Iteración asignada:** 3
Descripción: Como educador quiero visualizar las estadísticas de respuestas por jugador y por ODS para evaluar el aprendizaje.	
Observaciones: • Al finalizar la partida se debe mostrar una pantalla de estadísticas. • Debe incluir el porcentaje de respuestas correctas por jugador. • Debe identificar los ODS con menor porcentaje de aciertos para cada jugador. • Las estadísticas deben poder consultarse durante la partida mediante un botón de pausa.	

Tabla 2.13. Historia de usuario 11: Menú principal y pantalla de instrucciones (elaboración propia)

Historia de Usuario	
Número: 11	**Nombre:** Menú principal y pantalla de instrucciones
Prioridad: Alta	**Riesgo:** Baja
Puntos de estimación: 2.0	**Iteración asignada:** 1
Descripción: Como jugador quiero acceder a un menú principal intuitivo y a las instrucciones del juego para aprender a jugarlo fácilmente.	
Observaciones: • El menú principal debe incluir botones: Jugar, Instrucciones, Configuración y Salir. • La pantalla de instrucciones debe explicar la mecánica de forma visual. • Debe incluir referencias a los 17 ODS con sus colores e íconos oficiales.	

Tabla 2.14. Historia de usuario 12: Configuración de audio (elaboración propia)

Historia de Usuario	
Número: 12	**Nombre:** Configuración de audio
Prioridad: Baja	**Riesgo:** Baja
Puntos de estimación: 2.5	**Iteración asignada:** 4
Descripción: Como jugador quiero configurar el audio para personalizar mi experiencia durante la partida.	
Observaciones: • Debe incluir controles deslizantes para el volumen de música y de los efectos de sonido.	

Tabla 2.15. Historia de usuario 13: Personalización de jugadores (elaboración propia)

Historia de Usuario	
Número: 13	**Nombre:** Personalización de jugadores
Prioridad: Alta	**Riesgo:** Bajo
Puntos de estimación: 1.5	**Iteración asignada:** 1
Descripción: Como jugador quiero ingresar mi nombre y seleccionar el color de mi ficha para identificarme durante la partida.	
Observaciones: • Debe incluir un campo de texto para el nombre de cada jugador. • Debe ofrecer al menos 6 opciones de color para la ficha. • Los datos ingresados deben persistir durante toda la sesión. • El color seleccionado no debe estar disponible para los demás jugadores.	

Tabla 2.16. Historia de usuario 14: Visualización de información del ODS (elaboración propia)

Historia de Usuario	
Número: 14	**Nombre:** Visualización de información del ODS
Prioridad: Alta	**Riesgo:** Bajo
Puntos de estimación: 2.0	**Iteración asignada:** 2
Descripción: Como jugador quiero ver una breve descripción del ODS al aterrizar en su casilla para contextualizarme antes de responder la pregunta.	
Observaciones: • Al caer en casilla temática debe mostrarse el ícono oficial del ODS, su número, nombre y una descripción breve de no más de dos oraciones. • La información debe mostrarse antes de presentar la pregunta. • El diseño debe respetar los colores oficiales de cada ODS.	

Tabla 2.17. Historia de usuario 15: Temporizador de respuesta (elaboración propia)

Historia de Usuario	
Número: 15	**Nombre:** Temporizador de respuesta
Prioridad: Baja	**Riesgo:** Medio
Puntos de estimación: 2.0	**Iteración asignada:** 3
Descripción: Como educador quiero activar un temporizador opcional para que los jugadores respondan dentro de un límite de tiempo y agregar dinamismo a la partida.	
Observaciones: • El temporizador debe ser configurable antes de iniciar la partida (30, 60 o 90 segundos). • Al agotar el tiempo sin respuesta se debe contabilizar como respuesta incorrecta. • El temporizador debe mostrarse visualmente con cambio de color al llegar al 25 % del tiempo restante. • Su activación debe ser opcional.	

Tabla 2.18. Historia de usuario 16: Modo de un jugador (elaboración propia)

Historia de Usuario	
Número: 16	**Nombre:** Modo de un jugador
Prioridad: Media	**Riesgo:** Alto
Puntos de estimación: 5.0	**Iteración asignada:** 4
Descripción: Como jugador quiero poder jugar en solitario contra un oponente controlado por el sistema para practicar los ODS.	
Observaciones: • El modo un jugador debe ofrecer un oponente automático. • El oponente debe simular una velocidad de juego normal. • Las respuestas del oponente pueden ser correctas en un porcentaje configurable para ajustar la dificultad. • Debe diferenciarse visualmente el turno del jugador y el del sistema.	

Tabla 2.19. Historia de usuario 17: Pantalla de clasificación al finalizar (elaboración propia)

Historia de Usuario	
Número: 17	**Nombre:** Pantalla de clasificación al finalizar
Prioridad: Media	**Riesgo:** Bajo
Puntos de estimación: 1.5	**Iteración asignada:** 3
Descripción: Como jugador quiero ver una tabla clasificatoria al final de la partida para conocer el desempeño de todos los jugadores.	
Observaciones: • La pantalla de clasificación debe mostrar el puesto, nombre y porcentaje de respuestas correctas de cada jugador. • Debe destacar al ganador. • Debe incluir un botón de jugar de nuevo y uno de volver al menú principal. • Deve mostrarse animación de confeti para el primer lugar.	

Tabla 2.20. Historia de usuario 18: Notificación visual del turno activo (elaboración propia)

Historia de Usuario	
Número: 18	**Nombre:** Notificación visual del turno activo
Prioridad: Alta	**Riesgo:** Bajo
Puntos de estimación: 1.0	**Iteración asignada:** 2
Descripción: Como jugador quiero recibir una notificación visual y sonora de cuándo es mi turno para estar atento al juego.	
Observaciones: • Debe mostrarse un mensaje de “¡Es tu turno, [nombre]!” con animación de entrada. • La ficha del jugador activo debe resaltarse visualmente en el tablero. • Debe reproducirse un efecto de sonido al inicio de cada turno. • El botón de lanzar dado solo debe estar activo en el turno del jugador correspondiente.	

Tabla 2.21. Historia de usuario 19: Accesibilidad de la interfaz (elaboración propia)

Historia de Usuario	
Número: 19	**Nombre:** Accesibilidad de la interfaz
Prioridad: Media	**Riesgo:** Bajo
Puntos de estimación: 2.0	**Iteración asignada:** 5
Descripción: Como jugador quiero que la interfaz sea accesible y legible para poder disfrutar el juego sin dificultades visuales.	
Observaciones: • El tamaño de fuente en todos los textos del juego no debe ser inferior a 16pt. • Los botones deben tener un tamaño mínimo de 60x60 píxeles. • Las preguntas deben poder leerse sin necesidad de scroll. • La aplicación debe funcionar correctamente en resoluciones de 1024x768 o superiores.	

Tabla 2.22. Historia de usuario 20: Pantalla de créditos (elaboración propia)

Historia de Usuario	
Número: 20	**Nombre:** Pantalla de créditos
Prioridad: Baja	**Riesgo:** Bajo
Puntos de estimación: 0.5	**Iteración asignada:** 5
Descripción: Como jugador quiero acceder a una pantalla de créditos para conocer quiénes desarrollaron el juego.	
Observaciones: • La pantalla de créditos debe ser accesible desde el menú principal. • Debe mostrar el nombre del desarrollador, la institución y el año. • Debe incluir referencias a Go Goals! de las Naciones Unidas y a Godot Engine.	

Tabla 2.23. Historia de usuario 21: Exportación multiplataforma de escritorio (elaboración propia)

Historia de Usuario	
Número: 21	**Nombre:** Exportación multiplataforma de escritorio
Prioridad: Alta	**Riesgo:** Medio
Puntos de estimación: 3.0	**Iteración asignada:** 5
Descripción: Como educador quiero que el videojuego esté disponible en diversos sistemas operativos de escritorio para usarlo en cualquier aula sin restricciones de plataforma.	
Observaciones: • El videojuego debe exportarse correctamente a ejecutables nativos mediante Godot. • Debe funcionar en Windows, Linux y macOS. • Los requisitos de hardware deben ser mínimos para funcionar en equipos antiguos. • El ejecutable debe ser fácilmente distribuible mediante memorias USB u otros medios locales.	

2.2.4. Estimación de tiempo de desarrollo

Los puntos de estimación en el desarrollo ágil se utilizan para medir el esfuerzo relativo requerido para completar una historia de usuario, en lugar de estimar el tiempo exacto. Esto sirve para evaluar su complejidad, comparar historias de usuario y planificar las iteraciones (Cohn, 2020).

Tabla 2.24. Puntos de estimación por historia de usuario (elaboración propia)

No. HU	Nombre	Puntos (días)
1.	Inicio de nueva partida educativa	2.0
2.	Visualización del tablero hexagonal	5.0
3.	Lanzamiento de dado virtual	1.5
4.	Movimiento animado de ficha	4.0
5.	Efecto de escalera y serpiente	3.0
6.	Preguntas educativas sobre ODS	5.0
7.	Evaluación y retroalimentación de respuestas	3.0
8.	Gestión de turnos automática	2.0
9.	Condición de victoria	1.5
10.	Estadísticas de aprendizaje	4.0
11.	Menú principal y pantalla de instrucciones	2.0
12.	Configuración de audio	2.5
13.	Personalización de jugadores	1.5
14.	Visualización de información del ODS	2.0
15.	Temporizador de respuesta	2.0
16.	Modo de un jugador	5.0
17.	Pantalla de clasificación al finalizar	1.5
18.	Notificación visual del turno activo	1.0
19.	Accesibilidad de la interfaz	2.0
20.	Pantalla de créditos	0.5
21.	Exportación multiplataforma de escritorio	3.0
	Total	**53.5**

2.2.5. Plan de iteraciones

El plan de iteraciones es una herramienta esencial para organizar y coordinar el trabajo del equipo de desarrollo. Cada iteración suele durar entre una y tres semanas, y comienza con una reunión de planificación en la que el cliente presenta las historias de usuario priorizadas, y el equipo de desarrollo estima y se compromete a completar las tareas necesarias (Cohn, 2020). Este enfoque permite entregar incrementos funcionales del software de manera continua, asegurando que el producto evolucione de acuerdo con las necesidades cambiantes del cliente.

Tabla 2.25. Plan de iteraciones de desarrollo (elaboración propia)

Iteración	No. HU	Historia de Usuario	Duración (días)
1	1	Inicio de nueva partida educativa	10.5
	2	Visualización del tablero hexagonal	
	11	Menú principal y pantalla de instrucciones	
	13	Personalización de jugadores	
2	3	Lanzamiento de dado virtual	12.5
	4	Movimiento animado de ficha	
	14	Visualización de información del ODS	
	18	Notificación visual del turno activo	
3	5	Efecto de escalera y serpiente	11.5
	8	Gestión de turnos automática	
	9	Condición de victoria	
	10	Estadísticas de aprendizaje	
4	6	Preguntas educativas sobre ODS	10.0
	7	Evaluación y retroalimentación de respuestas	
	15	Temporizador de respuesta	
5	12	Configuración de audio	9.0
	16	Modo de un jugador	
	17	Pantalla de clasificación al finalizar	
	19	Accesibilidad de la interfaz	
	20	Pantalla de créditos	
	21	Exportación multiplataforma de escritorio	
		Total	**53.5**

2.3. Diseño

El presente epígrafe constituye la transición esencial entre la planificación teórica y la implementación práctica, donde se materializan los requisitos en soluciones técnicas concretas. Este apartado se centra en tres pilares fundamentales: (1) la arquitectura del sistema, detallando cómo la estructura basada en nodos y el SceneTree de Godot permiten organizar los componentes clave del videojuego; (2) los prototipos de interfaces gráficas de usuario; y (3) las tarjetas CRC (Class-Responsibility-Collaboration) que definen la interacción entre las clases del sistema.

2.3.1. Arquitectura utilizada

Godot Engine utiliza una arquitectura jerárquica fundamentada en el concepto de nodos como unidades básicas de construcción. Cada elemento funcional del videojuego es representado como un nodo que hereda de la clase base `Node` (Godot Engine Documentation, 2023). Esta filosofía arquitectónica de composición sobre herencia permite construir funcionalidades sofisticadas mediante la agregación de componentes simples y bien definidos. El videojuego *Go Goals! Digital* emplea una arquitectura modular orientada a escenas (*scene-driven*), apoyada por servicios globales y coordinada centralmente. Este enfoque estructura el proyecto en cuatro pilares fundamentales de diseño y responsabilidad.

El primer pilar es la **Arquitectura Orientada a Escenas**. La navegación principal y los límites de las interfaces dependen del sistema nativo de escenas de Godot. Entornos como el menú principal, la pantalla de juego interactiva y el menú de fin de partida funcionan como fronteras naturales de presentación. Dentro del juego, el nivel principal actúa como controlador de composición local (*composition root*), ensamblando las dependencias y conectando la interfaz de usuario con la lógica sin mezclar directamente sus responsabilidades.

El segundo pilar son los **Servicios Globales y Autoloads**. Godot facilita el patrón Singleton a través de nodos que se registran globalmente, permitiendo el acceso a funcionalidades transversales desde cualquier escena. El proyecto emplea la clase `GameData` para mantener la configuración de la partida y cargar el banco de preguntas; `AudioManager` para centralizar la reproducción de música y efectos de sonido; y `RecordsManager` para gestionar la persistencia local de las puntuaciones de forma estructurada e independiente (ibíd.).

El tercer pilar es la **Coordinación Central y Comunicación por Eventos**. En lugar de un patrón *Model-View-Controller* estricto, el flujo del juego es orquestado por un coordinador central de aplicación (`GameManager`). Este componente concentra los casos de uso principales: control de turnos, movimiento, preguntas y victoria. Para lograr un bajo acoplamiento, la comunicación hacia la interfaz gráfica se rige por el uso de señales o *Event Bus*. Así, interfaces como el panel de preguntas reaccionan de manera independiente a la aparición de eventos sin depender en código de la lógica del coordinador.

El cuarto pilar es el **Modelo de Estado Separado y Entidades Ligeras**. El estado mutable de la partida (fase del juego, turno actual, cantidad de tiradas) se aísla en un modelo de estado explícito (`GameState`), lo cual facilita administrar pausas o finalizaciones. Adicionalmente, el dominio se representa a través de entidades interactivas ligeras (elementos lógicos de tablero, casillas y fichas) que encapsulan su comportamiento particular, pero que delegan el control de flujo global al coordinador de la partida.

Estructura de directorios del proyecto

La organización del código fuente sigue una estructura jerárquica por responsabilidad funcional, coherente con los principios de alta cohesión y bajo acoplamiento (Pressman y Maxim, 2021):

- **autoloads/**: contiene los Singletons globales del proyecto (`GameData`, `AudioManager`, `RecordsManager`).

[Figura: SceneTree del videojuego Go Goals! Digital (elaboración propia)]

Figura 2.2. SceneTree y Autoloads del videojuego *Go Goals! Digital (elaboración propia)* en Godot Engine (elaboración propia)

- **scripts/Core/**: contiene los archivos de infraestructura del ciclo de vida del juego (`Events.gd` — Bus de Eventos —, `GameState.gd` — Máquina de Estados).
- **scripts/Data/**: contiene los archivos de configuración estática del tablero (`BoardConfig.gd`), donde se definen todas las casillas de escaleras, toboganes y preguntas con sus posiciones de origen y destino.
- **scripts/Entities/**: contiene los objetos del dominio del juego (`Board.gd`, `Player.gd`, `Tile.gd`), que encapsulan propiedades pero no decisiones.
- **scripts/Managers/**: contiene el `GameManager.gd`, controlador central que orquesta el ciclo de la partida.
- **scripts/UI/**: contiene los scripts de la capa de presentación, incluyendo los paneles del tablero, el panel de preguntas, el menú de pausa y la pantalla de estadísticas.

[Figura: Diagrama de la arquitectura multi-patrón de Go Goals! Digital (elaboración propia)]

Figura 2.3. Arquitectura multi-patrón del videojuego educativo *Go Goals! Digital (elaboración propia)* (elaboración propia)

Flujo de ejecución del ciclo de juego

El ciclo principal de una partida sigue un flujo bien definido y orquestado por el `GameManager`. Primero, durante la **inicialización**, el `GameManager` lee la configuración del tablero desde `BoardConfig.gd` e inicializa las entidades `Board` y los jugadores (`Player`) con sus posiciones de salida. Segundo, al **lanzar el dado**, el jugador presiona el botón correspondiente, `GameManager` llama a `roll_dice()` y emite la señal global `dice_rolled` a través de `GameEvents`. Tercero, en el **movimiento**, `GameManager` calcula el destino y aplica un `Tween` en `_animate_movement()` para desplazar visualmente la ficha casilla por casilla, reproduciendo sonidos a través de `AudioManager`. Cuarto, en la **resolución de casilla**, al terminar el movimiento (`_on_movement_finished`), se evalúa la casilla mediante `_check_tile()`: si es una *casilla normal*, el turno pasa al siguiente jugador actualizando `GameState`; si es una *casilla especial*, se anima automáticamente el traslado al destino; si es *casilla de quiz*, se emite la señal `quiz_requested(ods_id)` y la UI muestra el panel modal con la pregunta leída desde `GameData`; si es la *casilla final (63)*, se emite `game_won` y se detiene la música (Godot Engine Documentation, 2023).

2.3.2. Prototipos de interfaces gráficas

En el presente epígrafe se presentan las principales interfaces gráficas del videojuego en su forma de prototipo, describiendo cada uno de sus elementos y las funcionalidades asociadas. El diseño de las interfaces gráficas prioriza la accesibilidad, comprensibilidad y atractivo visual para niños de ocho a diez años, manteniendo fidelidad a los colores e íconos oficiales de los diecisiete ODS.

[Figura: Prototipo de la pantalla principal de tablero (elaboración propia)]

Figura 2.4. Prototipo de la pantalla principal del tablero de juego (elaboración propia)

Panel de preguntas educativas

El panel de preguntas constituye la interfaz modal que se superpone al tablero durante las casillas temáticas. Este panel muestra el ícono y el nombre del ODS correspondiente, el texto de la pregunta y las opciones

de respuesta claramente diferenciadas mediante botones de gran tamaño. Su diseño prioriza la legibilidad para el grupo de edad objetivo mediante una tipografía de tamaño adecuado. Tras la selección de una respuesta, el panel muestra la retroalimentación (correcto/incorrecto) y una breve explicación pedagógica antes de continuar el turno.

[Figura: Prototipo del panel de preguntas ODS (elaboración propia)]

Figura 2.5. Prototipo del panel de preguntas educativas por ODS (elaboración propia)

Pantalla de estadísticas

La pantalla de estadísticas se muestra al finalizar la partida y permite acceder a ella durante el juego desde el menú de pausa. Muestra el resumen de respuestas correctas e incorrectas por jugador y un desglose por ODS para identificar las áreas de menor conocimiento. Esto permite al educador diagnosticar los objetivos que requieren mayor atención pedagógica.

[Figura: Prototipo de la pantalla de estadísticas de aprendizaje (elaboración propia)]

Figura 2.6. Prototipo de la pantalla de estadísticas de aprendizaje (elaboración propia)

2.3.3. Tarjetas CRC

Las tarjetas CRC (Class-Responsibility-Collaboration) constituyen una técnica fundamental en el diseño ágil, particularmente relevante en la metodología Extreme Programming. Son una herramienta efectiva

para la conceptualización y diseño de sistemas orientados a objetos, que facilita la identificación de las clases, sus responsabilidades y las colaboraciones necesarias entre ellas (Sommerville, 2021). A continuación se presentan las principales tarjetas CRC de *Go Goals! Digital*, correspondientes a las clases reales del proyecto:

Tabla 2.26. Tarjeta CRC GameData (Autoload) (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
GameData	- mantener la configuración de la partida (número de jugadores) - cargar y servir el banco de preguntas desde `questions.json` - proveer preguntas aleatorias por ODS sin repetición	GameManager BoardConfig

Tabla 2.27. Tarjeta CRC GameEvents (Bus de Eventos) (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
GameEvents	- definir las señales globales del sistema: `dice_rolled`, `player_moving`, `quiz_started`, `quiz_requested(ods_id)`, `game_won` - actuar como canal de comunicación desacoplado entre lógica y UI	GameManager Scripts de UI

Tabla 2.28. Tarjeta CRC GameState (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
GameState	- gestionar la fase activa del juego mediante enumeración (MENU, PLAYING, PAUSED, GAME_OVER) - almacenar el estado de la partida en curso: jugador activo (`active_player_index`), posiciones y conteo de turnos	GameManager Player

Tabla 2.29. Tarjeta CRC GameManager (Controlador) (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
GameManager	- orquestar el ciclo completo de la partida (inicio, turno, resolución de casilla, fin) - calcular rutas de movimiento e invocar animaciones - verificar condiciones de victoria	GameState GameEvents BoardConfig AudioManager

Tabla 2.30. Tarjeta CRC BoardConfig (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
BoardConfig	- definir la configuración estática del tablero: posiciones de escaleras, toboganes y casillas de pregunta por ODS - proveer información de tipo y destino de cada casilla	GameManager Board

Tabla 2.31. Tarjeta CRC Board (Entidad) (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
Board	- representar el estado visual y lógico del tablero de 63 casillas - exponer métodos de acceso a posiciones específicas	GameManager Tile BoardConfig

Tabla 2.32. Tarjeta CRC Tile (Entidad) (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
Tile	- mantener el tipo de casilla (normal, escalera, tobogán, quiz, meta) - almacenar el destino de traslado en caso de casilla especial - almacenar el identificador ODS en caso de casilla de pregunta	Board GameManager

Tabla 2.33. Tarjeta CRC Player (Entidad) (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
Player	- mantener la posición actual y el historial de turnos del jugador - almacenar nombre e identificador de color de ficha - registrar estadísticas de respuestas	GameState GameManager

Tabla 2.34. Tarjeta CRC AudioManager (Autoload) (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
AudioManager	- centralizar la reproducción de música de fondo y efectos de sonido - controlar el volumen global - liberar a los nodos de juego de gestionar `AudioStreamPlayer` locales	GameManager Scripts de UI

Tabla 2.35. Tarjeta CRC RecordsManager (Autoload) (elaboración propia)

Clase	**Responsabilidades**	**Colaboradores**
RecordsManager	- gestionar la persistencia de puntuaciones y logros entre sesiones - proveer datos históricos para la pantalla de estadísticas	GameManager Player

2.4. Implementación

El presente epígrafe describe el proceso de implementación del videojuego educativo *Go Goals! Digital*, abordando los estándares de codificación adoptados, los patrones de diseño aplicados y las tareas de ingeniería ejecutadas en cada iteración.

2.4.1. Estándares de codificación

Los estándares de codificación son un conjunto de convenciones y reglas que guían la forma en que se escribe el código fuente de un sistema, con el objetivo de mejorar su legibilidad, mantenibilidad y consistencia. Según Sommerville (Sommerville, 2021), el cumplimiento de estándares de codificación es un factor clave para reducir la deuda técnica y facilitar la colaboración en equipos de desarrollo.

Nomenclatura y estructura de archivos

Para el desarrollo en GDScript se aplicaron los siguientes estándares de nomenclatura, coherentes con las convenciones oficiales de la documentación de Godot (Godot Engine Documentation, 2023):

- **Clases**: PascalCase (ejemplo: `GameManager`, `QuestionManager`).
- **Variables y funciones**: snake_case (ejemplo: `current_player`, `roll_dice()`). **Constantes**: SCREAMING_SNAKE_CASE (ejemplo: `MAX_PLAYERS`, `BOARD_SIZE`).
- **Señales**: snake_case prefijadas con el evento (ejemplo: `dice_rolled`, `answer_checked`).
- **Escenas**: PascalCase coincidiendo con el nombre de su script raíz (ejemplo: `GameBoard.tscn`).

La estructura de archivos del proyecto sigue una organización jerárquica por funcionalidad:

[Figura: Estructura jerárquica de archivos del proyecto (elaboración propia)]

Figura 2.7. Estructura de archivos del proyecto *Go Goals! Digital (elaboración propia)* (elaboración propia)

Documentación del código

Cada clase implementada en GDScript incluye comentarios de documentación al inicio del archivo describiendo su propósito, y comentarios de bloque en funciones de lógica compleja. Esta práctica facilita el mantenimiento futuro del código y permite la incorporación de nuevos desarrolladores al proyecto (ibíd.).

2.4.2. Patrones de diseño utilizados

Los patrones de diseño son soluciones reutilizables a problemas recurrentes en el desarrollo de software. *Go Goals! Digital* aplica cuatro patrones principales que actuan de forma complementaria y se encuentran reflejados directamente en la estructura del código fuente.

Patrón Singleton mediante Autoloads

Este patrón GOF garantiza que determinadas clases tengan una única instancia accesible globalmente durante toda la sesión de juego (Pressman y Maxim, 2021). Godot implementa este patrón de forma nativa mediante el mecanismo de Autoloads. Los tres Autoloads del proyecto son: `GameData`, que actua como repositorio central de configuración y datos educativos; `AudioManager`, que centraliza toda la reproducción de audio evitando instancias dispersas de `AudioStreamPlayer`; y `RecordsManager`, que gestiona la persistencia de logros entre sesiones. Gracias a este patrón, cualquier nodo del árbol de escenas puede acceder a estos servicios sin crear dependencias directas ni referencias cruzadas entre escenas (Godot Engine Documentation, 2023).

Patrón Bus de Eventos (Observer global)

El archivo `GameEvents` implementa una variante del patrón Observer a nivel de bus de eventos global (Pressman y Maxim, 2021). En lugar de conectar directamente un emisor con sus suscriptores, `GameEvents` define señales globales que cualquier componente puede emitir o escuchar de forma independiente. Las señales principales del sistema son: `dice_rolled` (el dado fue lanzado), `player_moving` (la ficha está en movimiento), `quiz_started` (se inició una ronda de pregunta), `quiz_requested(ods_id)` (se solicita la pregunta de un ODS específico) y `game_won` (un jugador alcanzó la meta). Este patrón elimina el acoplamiento entre el `GameManager` y los componentes de UI: el controlador emite `quiz_requested(ods_id)` sin saber quién lo escucha, y la UI responde de forma independiente (Godot Engine Documentation, 2023).

[Figura: Bus de Eventos (GameEvents) — flujo de señales entre GameManager y UI (elaboración propia)]

Figura 2.8. Patrón Bus de Eventos implementado mediante señales globales en Godot (elaboración propia)

Patrón Máquina de Estados

El estado de la partida se gestiona de forma explícita y centralizada en `GameState.gd`, mediante una enumeración de fases: `MENU`, `PLAYING`, `PAUSED` y `GAME_OVER` (Pressman y Maxim, 2021). Esta separación del estado respecto a la lógica de actualización garantiza que ninguna transición de estado quede implícita o dispersa en el código. Además, `GameState` centraliza en memoria RAM el estado de la partida en curso: el índice del jugador activo (`active_player_index`), el historial de turnos y las posiciones actuales de

todas las fichas, facilitando las operaciones de guardado y restauración. La aplicación de este patrón permite al `GameManager` delegar las decisiones de transición de fase a una entidad especializada, reduciendo la complejidad ciclomática del controlador.

Patrón MVC orientado a componentes (GRASP)

La arquitectura del proyecto aplica los principios GRASP (General Responsibility Assignment Software Patterns) (Pressman y Maxim, 2021) en una estructura MVC adaptada al modelo de nodos de Godot. El **Controlador** es `GameManager.gd`, que centraliza la orquestación del ciclo de juego aplicando el principio GRASP de *Controlador de Fachada*. Los **Modelos** son las entidades `Board.gd`, `Player.gd` y `Tile.gd`, que encapsulan propiedades y aplican el principio de *experto en información* (cada entidad conoce solo sus propios datos). La **Vista** es gestionada por los scripts visuales y de interfaz, que escuchan eventos de `GameEvents` sin acoplarse al modelo.

Es notable que la configuración del tablero se aisla en `BoardConfig.gd`, separando los datos estáticos de escaleras, toboganes y casillas de quiz del código lógico, lo que permite modificar la configuración del tablero sin alterar el controlador. Esta decisión de diseño aplica directamente el principio GRASP de *bajo acoplamiento*, garantizando que el sistema sea fácilmente extensible ante futuras modificaciones del tablero o las reglas del juego (ibíd.).

2.4.3. Tareas de ingeniería

Las tareas de ingeniería son los componentes técnicos específicos derivados de las historias de usuario. Representan el trabajo concreto que debe realizarse para implementar cada funcionalidad. A continuación se presentan las tareas de ingeniería organizadas por iteración:

Tabla 2.36. Tarea de ingeniería 1: Diseño e implementación del tablero hexagonal (elaboración propia)

Campo	**Descripción**
Tarea #	1
HU relacionada	HU #2 - Visualización del tablero hexagonal
Nombre	Diseño e implementación del tablero hexagonal
Tipo	Desarrollo
Puntos estimados	3.0 días
Iteración	1
Programador	Ruslan Alexei Martinez Campos
Descripción	Se diseña el tablero en Inkscape con las 63 casillas hexagonales siguiendo la distribución oficial. Se implementa en Godot como TileMap y se configuran las propiedades de cada casilla (tipo, destino).

Tabla 2.37. Tarea de ingeniería 2: Implementación del menú principal (elaboración propia)

Campo	**Descripción**
Tarea #	2
HU relacionada	HU #11 - Menú principal y pantalla de instrucciones
Nombre	Implementación del menú principal y pantalla de instrucciones
Tipo	Desarrollo
Puntos estimados	1.5 días
Iteración	1
Programador	Ruslan Alexei Martinez Campos
Descripción	Se diseña la escena del menú principal con los botones Jugar, Instrucciones, Configuración y Salir. Se implementa la pantalla de instrucciones con la explicación visual de la mecánica del juego.

Tabla 2.38. Tarea de ingeniería 3: Lógica del dado y movimiento de fichas (elaboración propia)

Campo	**Descripción**
Tarea #	3
HU relacionada	HU #3, HU #4
Nombre	Implementación de la lógica del dado y movimiento animado de fichas
Tipo	Desarrollo
Puntos estimados	5.5 días
Iteración	2
Programador	Ruslan Alexei Martinez Campos
Descripción	Se implementa el `DiceController` con generación pseudoaleatoria y animación del dado. Se implementa el `PieceController` con movimiento casilla por casilla mediante `Tween` de Godot.

Tabla 2.39. Tarea de ingeniería 4: Escaleras, serpientes y gestión de turnos (elaboración propia)

Campo	**Descripción**
Tarea #	4
HU relacionada	HU #5, HU #8, HU #9
Nombre	Implementación de efectos especiales y gestión de turnos
Tipo	Desarrollo
Puntos estimados	6.5 días
Iteración	3
Programador	Ruslan Alexei Martinez Campos
Descripción	Se implementa la detección de casillas de escalera y serpiente en el `BoardManager`. Se implementa el `TurnManager` con la lógica de rotación de turnos y la condición de victoria al alcanzar la casilla 63.

Tabla 2.40. Tarea de ingeniería 5: Sistema de preguntas y evaluación de respuestas (elaboración propia)

Campo	**Descripción**
Tarea #	5
HU relacionada	HU #6, HU #7
Nombre	Implementación del sistema de preguntas educativas y retroalimentación
Tipo	Desarrollo
Puntos estimados	8.0 días
Iteración	4
Programador	Ruslan Alexei Martinez Campos
Descripción	Se implementa el `QuestionManager` con carga de preguntas desde JSON y selección aleatoria sin repetición. Se diseña el panel de preguntas en Godot con el ícono ODS, el texto de la pregunta y los botones de respuesta. Se implementa la retroalimentación visual y sonora y la generación de efectos de partículas para respuestas correctas.

Tabla 2.41. Tarea de ingeniería 6: Estadísticas de aprendizaje (elaboración propia)

Campo	**Descripción**
Tarea #	6
HU relacionada	HU #10
Nombre	Implementación del sistema de estadísticas de aprendizaje
Tipo	Desarrollo
Puntos estimados	6.5 días
Iteración	5
Programador	Ruslan Alexei Martinez Campos
Descripción	Se implementa el `StatsManager` como autoload para registrar respuestas durante la sesión. Se diseña la pantalla de estadísticas con visualización por jugador y por ODS.

Tabla 2.42. Tarea de ingeniería 7: Personalización de jugadores (elaboración propia)

Campo	**Descripción**
Tarea #	7
HU relacionada	HU #13
Nombre	Sistema de personalización de perfiles de jugador
Tipo	Desarrollo
Puntos estimados	1.5 días
Iteración	1
Programador	Ruslan Alexei Martinez Campos
Descripción	Diferenciación visual de fichas. Implementación de la persistencia de nombres y colores seleccionados a través de `GameData`. Validación de colores para evitar duplicados.

Tabla 2.43. Tarea de ingeniería 8: Panel informativo de ODS (elaboración propia)

Campo	**Descripción**
Tarea #	8
HU relacionada	HU #14
Nombre	Implementación del panel informativo previo a preguntas
Tipo	Desarrollo
Puntos estimados	2.0 días
Iteración	2
Programador	Ruslan Alexei Martinez Campos
Descripción	Creación de interfaz informativa que muestra íconos y objetivos específicos antes de la fase de evaluación. Sincronización con el Bus de Eventos para pausar el flujo de juego.

Tabla 2.44. Tarea de ingeniería 9: Notificaciones de turno activo (elaboración propia)

Campo	**Descripción**
Tarea #	9
HU relacionada	HU #18
Nombre	Sistema de resaltado y notificación de turno
Tipo	Desarrollo
Puntos estimados	1.0 días
Iteración	2
Programador	Ruslan Alexei Martinez Campos
Descripción	Implementación de efectos de brillo (glow) sobre la ficha del jugador activo y banners animados de entrada de turno. Integración con señales de `GameState`.

Tabla 2.45. Tarea de ingeniería 10: Inteligencia Artificial oponente (elaboración propia)

Campo	**Descripción**
Tarea #	10
HU relacionada	HU #16
Nombre	Implementación de lógica para oponente virtual (IA)
Tipo	Desarrollo
Puntos estimados	5.0 días
Iteración	5
Programador	Ruslan Alexei Martinez Campos
Descripción	Desarrollo de un autómata que simula la toma de decisiones y el lanzamiento de dados. Configuración de probabilidades de acierto para ajustar la dificultad del juego.

Tabla 2.46. Tarea de ingeniería 11: Exportación y optimización multiplataforma (elaboración propia)

Campo	**Descripción**
Tarea #	11
HU relacionada	HU #21
Nombre	Optimización y despliegue multiplataforma (HTML5)
Tipo	Desarrollo
Puntos estimados	3.0 días
Iteración	5
Programador	Ruslan Alexei Martinez Campos
Descripción	Configuración de perfiles de exportación en Godot para Windows, Linux y macOS. Optimización de assets para reducir el peso final de los instaladores. Verificación de compatibilidad en diferentes resoluciones de pantalla nativas.

Tabla 2.47. Tarea de ingeniería 12: Configuración de sistema y accesibilidad (elaboración propia)

Campo	**Descripción**
Tarea #	12
HU relacionada	HU #12, HU #19
Nombre	Ajustes de audio y parámetros de legibilidad visual
Tipo	Desarrollo
Puntos estimados	4.5 días
Iteración	5
Programador	Ruslan Alexei Martinez Campos
Descripción	Implementación de buses de audio centralizados y paneles de configuración. Ajuste dinámico de escala de fuentes y tamaños de botones para garantizar accesibilidad.

Conclusiones del capítulo

A partir del proceso de análisis, diseño e implementación descrito en el presente capítulo, se establecen las siguientes conclusiones:

1. La solución propuesta constituye una adaptación digital fiel del juego de mesa *Go Goals!* que preserva su esencia pedagógica mientras incorpora capacidades tecnológicas que superan sus limitaciones estructurales, respondiendo directamente al problema científico planteado.
2. Los artefactos de ingeniería de requisitos obtenidos mediante 21 historias de usuario reflejan de manera precisa las necesidades pedagógicas y funcionales del videojuego, garantizando la alineación entre los objetivos educativos y las decisiones técnicas adoptadas.
3. La arquitectura en capas modulares implementada en Godot Engine, fundamentada en el SceneTree y el sistema de señales, favorece la mantenibilidad, escalabilidad y extensibilidad del sistema, permitiendo futuras incorporaciones de contenido y funcionalidades sin afectar los componentes existentes.
4. La separación entre contenido educativo y código lógico mediante archivos JSON facilita la actualización del banco de preguntas por parte de educadores sin conocimientos de programación, potenciando la sostenibilidad del recurso como herramienta pedagógica colaborativa.
5. La aplicación de patrones de diseño GRASP y GOF, junto con estándares de codificación coherentes con las convenciones de GDScript, garantiza la calidad técnica del software y sienta las bases para su validación en el capítulo siguiente.

CAPÍTULO 3

Resultados

Escribir aquí el capítulo 3. Otra prueba más engine.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Conclusiones

Escribir aquí las conclusiones.

Recomendaciones

Escribe aquí las recomendaciones de tu trabajo.

Referencias bibliográficas

BANCO MUNDIAL, 2023. *Indicadores de Desarrollo Mundial: Educación en Cuba* [https://datos.bancomundial.org/indicador/SE.ADT.LITR.ZS?locations=CU] (vid. págs. 2, 9).

BANCO MUNDIAL, 2025. *Programa de Educación para América Latina y el Caribe* [https://www.bancomundial.org/es/programs/educacion-america-latina-caribe] (vid. pág. 1).

BECK, Kent, 1999. *Extreme Programming Explained: Embrace Change*. Addison-Wesley (vid. págs. 13, 22).

CHACON, Scott y STRAUB, Ben, 2014. *Pro Git* [https://git-scm.com/book/en/v2]. 2.ª ed. Apress (vid. pág. 17).

CHEN, Jason; SMITH, Thomas y ALVAREZ, Roberto, 2022. Adaptive difficulty in video games: A machine learning approach. *IEEE Transactions on Games*. Vol. 15, n.º 2, págs. 1-14 (vid. pág. 5).

COCKBURN, Alistair, 2020. *Agile Software Development: The Cooperative Game*. 2.ª ed. Addison-Wesley (vid. pág. 13).

COHN, Mike, 2020. *Agile Estimating and Planning*. 2.ª ed. Prentice Hall (vid. págs. 24, 35, 36).

COMISIÓN MUNDIAL SOBRE EL MEDIO AMBIENTE Y EL DESARROLLO, 1987. *Nuestro Futuro Común (Informe Brundtland)*. Naciones Unidas. Documento ONU A/42/427 (vid. págs. 1, 5).

DETERDING, Sebastian; ZAGAL, José P. y BJÖRK, Staffan, 2020. Game design elements in non-game contexts: A framework for gamification. *Proceedings of the ACM on Human-Computer Interaction*. Vol. 4, n.º CSCW2, págs. 1-22 (vid. pág. 6).

ECOLOGÍAVERDE, 2024. *Ventajas y desventajas del desarrollo sostenible* [https://www.ecologiaverde.com/ventajas-y-desventajas-del-desarrollo-sostenible-319.html] (vid. pág. 2).

EUROPEAN COUNCIL, 2022. *Digital education action plan 2021-2027: Resetting education and training for the digital age* [https://education.ec.europa.eu/sites/default/files/document-library-docs/deap-communication-sept2020_en.pdf] (vid. pág. 6).

EUROPEAN UNION, 2024. *Sustainable development in the European Union: Overview of progress towards the SDGs* [https://ec.europa.eu/eurostat/web/sdi] (vid. pág. 1).

FLOOD, Stephen; CRADOCK-HENRY, N. A. y BLACKETT, P., 2024. A collaborative adaptation game for promoting climate action: Minions of Disruptions™. *Geoscience Communication*. Vol. 7, n.º 1, págs. 167-189. Disp. desde DOI: 10.5194/gc-7-167-2024 (vid. págs. 2, 6).

FOWLER, Martin, 2018. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3.ª ed. Addison-Wesley (vid. pág. 14).

GITHUB, INC., 2022. *GitHub Projects documentation* [https://docs.github.com/en/github/managing-your-work-on-github/about-projects] (vid. pág. 18).

GLOBAL EDUCATION MONITORING REPORT TEAM, 2022. *Global Education Monitoring Report 2021/2: Non-State Actors in Education: Who Chooses? Who Loses?* [https://en.unesco.org/gem-report/non-state_actors]. UNESCO (vid. pág. 1).

GODOT ENGINE, 2023. *Godot 4.4.1 – Motor de videojuegos de código abierto* [https://godotengine.org] (vid. págs. 15, 21).

GODOT ENGINE DOCUMENTATION, 2023. *Documentación Oficial de Godot Engine 4* [https://docs.godotengine.org] (vid. págs. 16, 17, 38, 40, 45, 46).

GREGORY, Jason, 2020. *Game Engine Architecture*. 3.ª ed. CRC Press (vid. pág. 15).

GRUPO NACIONAL PARA LA IMPLEMENTACIÓN DE LA AGENDA 2030, 2021. *Informe Nacional Voluntario Cuba 2021 sobre la Agenda 2030* [https://www.mep.gob.cu/es/documento/informe-nacional-voluntario-cuba-2021-sobre-la-agenda-2030] (vid. págs. 2, 9, 21).

HAWLITSCHEK, Florian; JOECKEL, Sven y WITTSTOCK, Matthias, 2022. Gameful Learning for a More Sustainable World. *Business & Information Systems Engineering*. Vol. 64, n.º 4, págs. 459-482. Disp. desde DOI: 10.1007/s12599-021-00731-x (vid. pág. 6).

JOHNSON, David y LEE, Kevin, 2021. The psychology of game design: How feedback loops shape player behavior. *Computers in Human Behavior*. Vol. 114, pág. 106557 (vid. pág. 5).

LINIETSKY, Juan y MANZUR, Ariel, 2020. *Godot Engine: The Complete Guide to Creating Games with Godot*. Packt Publishing (vid. págs. 15, 16).

MARTÍNEZ, L.; GIMENES, M. y LAMBERT, E., 2023. Video games and board games: Effects of playing practice on cognition. *PLoS ONE*. Vol. 18, n.º 3, e0283654. Disp. desde DOI: 10.1371/journal.pone.0283654 (vid. págs. 2, 7).

MESA-LAGO, Carmelo, 2022. *La economía cubana: Entre el embargo y la pandemia*. Vol. 42. N.º 1 (vid. págs. 2, 9).

NACIONES UNIDAS, 2015. *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible* [https://www.un.org/sustainabledevelopment/es/]. Resolución A/RES/70/1 (vid. págs. 1, 5, 20).

NACIONES UNIDAS, 2020. *Go Goals! Board Game* [https://www.un.org/sustainabledevelopment/es/takeaction/]. Official educational board game by the United Nations (vid. págs. 2, 8, 11, 12, 20).

PLASS, Jan L.; HOMER, Bruce D. y KINZER, Charles K., 2020. Foundations of game-based learning. *Educational Psychologist*. Vol. 50, n.º 4, págs. 258-283. Disp. desde DOI: 10.1080/00461520.2015.1122533 (vid. págs. 4-6).

PLASS, Jan L.; MAYER, Richard E. y HOMER, Bruce D., 2020. *Handbook of Game-Based Learning*. MIT Press. Disp. desde DOI: 10.7551/mitpress/11864.001.0001 (vid. pág. 5).

PRESSMAN, Roger S. y MAXIM, Bruce R., 2021. *Software Engineering: A Practitioner's Approach*. 9.ª ed. McGraw-Hill Education (vid. págs. 13, 22, 23, 38, 46, 47).

PROGRAMA DE LAS NACIONES UNIDAS PARA EL DESARROLLO, 2021. *Informe sobre Desarrollo Humano 2021/2022: Tiempos inciertos, vidas inestables* [https://hdr.undp.org/informe-sobre-desarrollo-humano-2021-22] (vid. pág. 5).

R3CREA, 2022. *Ventajas y desventajas de los objetivos de desarrollo sostenible* [https://r3crea.com/innovacion/ventajas-y-desventajas-de-los-objetivos-de-desarrollo-sostenible/] (vid. pág. 2).

REPÚBLICA DE CUBA, 2019. *Constitución de la República de Cuba*. Artículo 73: Derecho a la educación. Gaceta Oficial No. 5 Extraordinaria (vid. págs. 1, 9).

ROGERS, Scott, 2020. *Level Up! The Guide to Great Video Game Design*. 3.ª ed. Wiley (vid. pág. 5).

SCHWABER, Ken y SUTHERLAND, Jeff, 2020. *The Scrum Guide: The definitive guide to Scrum: The rules of the game* [https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf] (vid. págs. 13, 22).

SOMMERVILLE, Ian, 2021. *Software Engineering*. 10.ª ed. Pearson Education (vid. págs. 13, 14, 16, 42, 44).

STIGLITZ, Joseph E.; SEN, Amartya y FITOUSSI, Jean-Paul, 2018. *Report by the Commission on the Measurement of Economic Performance and Social Progress* [https://www.economie.gouv.fr/files/finances/presse/dossiers_de_presse/090914mesure_perf_eco_progres_social/synthese_ang.pdf] (vid. pág. 1).

UNESCO, 2017. *Educación para el Desarrollo Sostenible: Objetivos de aprendizaje* [https://www.unesco.org/es/articles/educacion-para-el-desarrollo-sostenible] (vid. pág. 1).

UNESCO, 2020. *Education for Sustainable Development: A Roadmap* [https://www.unesco.org/es/articles/educacion-para-el-desarrollo-sostenible-hoja-de-ruta] (vid. págs. 1, 5).

UNESCO, 2023. *Los adolescentes tienen el doble de probabilidad de no ir a la escuela que los niños en edad escolar* [https://www.unesco.org/es/articles/los-adolescentes-tienen-el-doble-de-probabilidad-de-no-ir-la-escuela-que-los-ninos-en-edad-escolar] (vid. págs. 1, 9).

UNICEF, 2020. *La accesibilidad de la educación a distancia: Un informe sobre la brecha digital durante el COVID-19* [https://coronavirus.onu.org.mx/al-menos-una-tercera-parte-de-los-ninos-en-edad-escolar-en-el-mundo-no-tuvo-acceso-a-educacion-a-distancia-durante-el-cierre-de-las-escuelas-por-covid-19] (vid. pág. 1).

VISUAL PARADIGM, 2023. *Visual Paradigm: The all-in-one CASE tool for software development* [https://www.visual-paradigm.com/] (vid. pág. 14).

WU, Jason S. y LEE, Joey J., 2020. Climate change games as tools for education and engagement. *Nature Climate Change*. Vol. 10, n.º 5, págs. 526-532. Disp. desde DOI: 10.1038/s41558-020-0740-1 (vid. págs. 2, 6).

Generado con LaTeX: 6 de marzo de 2026: 2:56am

Apéndices

APÉNDICE A

Proyectos y Avales

REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

1

APÉNDICE B

Otro reporte

B.1. Otra sección de muestra

De una sola sentencia

B.2. Otro ensamble Industrial

REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

1

REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Hormiga: 7
Reorientaciones: 2

Salida:
(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)-(3, -z)

Hormiga: 8
Reorientaciones: 2

Salida:
(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(12, -z)-(11, -z)

Hormiga: 9
Reorientaciones: 3

Salida:
(8, -x)-(14, -x)-(15, -x)-(16, -x)-(10, -z)-(11, -z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(17, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(12, -x)

Hormiga: 10
Reorientaciones: 2

Salida:
(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)

Hormiga: 11
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)

Secuencia de salida premiada
Reorientaciones: 2

Salida:
(8, -x)-(15, -x)-(14, -x)-(16, -x)-(10, -z)-(11, -z)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(6, +z)-(13, +z)-(3, +z)-(17, +z)-(2, +z)-(1, +z)